

華中科技大學

本科生毕业设计

采用元数据键值对存储的文件操作设计与实现

院 系 计算机科学与技术

专业班级 计科 2104

姓 名 岳云鹏

学 号 U202112345

指导教师 郭德纲

2025 年 13 月 32 日

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名： 2025 年 13 月 32 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保 密 ☐，在 年解密后适用本授权书

2、不保密 ☒。

(请在以上相应方框内打“√”)

作者签名： 2025 年 13 月 32 日

导师签名： 2025 年 13 月 32 日

摘要

文件系统中元数据操作（创建、删除、重命名等）通常占到 I/O 负载的 60%–80%，元数据的组织方式对系统整体吞吐有直接影响。传统文件系统使用 B+ 树管理目录项和 inode 索引，操作粒度为磁盘页——即便只增删一条目录项，也需要将整页读出、修改后再写回。在高并发小文件负载下，这种就地修改还会与目录级的锁竞争叠加，写放大和并发受限两个问题同时出现。键值（Key-Value, KV）存储把操作粒度从页降到单条记录，每条目录项独立保存为一个键值对，增删只需一次写入，不再涉及页结构的维护。在此基础上，基于 LSM-tree（Log-Structured Merge-tree）的 KV 引擎把分散的小写聚集到内存 MemTable 中，待其写满后再顺序批量刷盘，将元数据常见的随机小写转为顺序写入。

基于上述思路，本课题设计并实现了 RucksFS——一个以 RocksDB 为存储引擎、通过 FUSE 提供 POSIX 接口的用户态文件系统，覆盖 `create`、`mkdir`、`unlink`、`rmdir`、`rename`、`readdir`、`setattr` 等常见文件操作。在键值存储上实现这些操作需要回答三个设计问题：元数据如何建模为键值结构、父目录的写放大如何控制、并发访问下的语义正确性如何保证。

存储建模上，系统按列族（Column Family, CF）划分数据：inode 属性和目录项各占一个独立列族，配以不同的布隆过滤器和压缩策略。目录项列族的键以「父目录 inode 号 + 子文件名」大端序拼接构成，每条目录项对应目录树中的一条父子边。文件操作由此转化为对边的增删——`create` 插入一条边，`unlink` 删除一条边，`rename` 删除旧边后再插入新边——复杂度均为常数时间。相比之下，以全路径作为键的方案在执行 `rename` 时需要递归改写子树中的每一条键，代价随子树规模线性增长。此外，同一父目录下的所有子边在键空间中相邻，目录遍历与单项查找可分别通过前缀扫描和点查完成。

写放大方面，POSIX 语义要求 `create` 和 `unlink` 同步更新父目录的时间戳与链接计数。高并发下同一父目录会反复成为写热点，沿用读-改-写的方式难以避免长时间的锁等待。本课题的做法是引入增量更新与懒折叠机制（DeltaOp）：写入时追加一条属性增量记录，不改动基础 inode 值，将就地改写转为追加写入；读取时把基础值与未合并的增量实时叠加后返回，保证调用方看到的是最新状

态；后台线程定期扫描累积的增量，当条数超过阈值时一次折叠回基础值，从而在控制读路径开销的同时消除父目录的写竞争。

并发安全方面，多线程对同一目录执行 `create`、`rmdir`、`rename` 等操作时，检查与修改之间存在天然的竞态窗口：`create` 需先确认无同名项再插入，`rmdir` 需先确认目录为空再删除，`rename` 还涉及跨目录的多键原子修改。本课题采用 RocksDB 的悲观并发控制（Pessimistic Concurrency Control, PCC）事务，对涉及的键逐一加锁，将检查与修改封装在同一原子操作中，消除上述竞态。

在分布式部署（客户端与 MetadataServer 位于不同节点）下运行 `pjdfstest` 全部 398 个用例。其中，167 个用例依赖本文明确不实现的 `mknod` 和 `mkfifo` 接口，1 个用例超出 DataServer 当前 64 MiB 的单文件开发上限，另有 48 个用例被 `pjdfstest` 框架自身跳过（如 `utimensat`、`remount` 等，与元数据设计无关）。余下 182 个用例构成本文的实现范围，全部通过，通过率 100%。

性能方面，测试集群由 1 台 64 核服务器与最多 96 台 2 核客户端组成，使用 `mdtest` 压测。在 $N \in \{2, 8, 32\}$ 三档并发下对 RucksFS、NFS v4.2 与 JuiceFS+TiKV 作三方对比。相对 JuiceFS+TiKV，RucksFS 在 `create`、`remove`、`stat` 上分别领先约 1.5–1.7、1.8–2.2 和 1.2 倍，原因一是单机直连 RocksDB 不需承担 TiKV 的 Raft 与两阶段提交开销，二是 DeltaOp 缓解了共享父目录的写竞争。相对 NFS v4.2， $N = 2$ 时 RucksFS 因关闭属性缓存而每次 `create` 多走两次 lookup RPC，略慢于 NFS； $N = 32$ 时 NFS 受限于 ext4 的 `i_rwsem` 锁，RucksFS 在 `create` 上反超约 4.6 倍。

$N \geq 64$ 时，NFS 和 JuiceFS+TiKV 均无法稳定给出数据。在此区间内，对比对象转为 RucksFS 自身的 Delta 和 NoDelta 两个构建变体，按 $np = N \times r$ 扫描并发。当 np 超过 96 后，NoDelta 因反复的事务冲突无法继续增长，稳态吞吐保持在 12 000 ops/s 附近；Delta 则继续扩展至 38 000 ops/s， $np = 384$ 时 `create` 与 `remove` 的比值分别为 2.96 和 2.63。在 easy 模式（每个进程使用独立父目录）下两个变体基本一致，说明上述收益来自消除共享父目录的写冲突。

关键词： 文件系统；元数据管理；键值存储；LSM-tree；增量更新；悲观并发控制；FUSE

Abstract

Metadata operations—file creation, deletion, renaming, and the like—typically account for 60%–80% of file system I/O, which makes the organisation of metadata a direct determinant of overall throughput. Traditional file systems manage directory entries and inode indexes with B+ trees at page granularity: even a single-entry change forces a page read, modification, and write-back. Under high-concurrency small-file workloads, this in-place update is further compounded by directory-level lock contention, so write amplification and concurrency bottlenecks arise at the same time. Key-Value (KV) storage lowers the operational granularity from a page to a single record—each directory entry is stored as an independent key-value pair and an insertion or deletion reduces to one write, eliminating the need to maintain page-level structures. On top of this, KV engines based on the Log-Structured Merge-tree (LSM-tree) aggregate scattered small writes in an in-memory MemTable and, once it fills, flush them to disk in sequential batches, turning the random small writes common in metadata workloads into sequential ones.

Building on this idea, this work designs and implements RucksFS, a user-space file system backed by RocksDB and exposed as a POSIX mount via FUSE, covering the common file operations `create`, `mkdir`, `unlink`, `rmdir`, `rename`, `readdir`, and `setattr`. Implementing these operations over KV storage requires answering three design questions: how to model metadata as key-value structures, how to control write amplification on parent directories, and how to guarantee semantic correctness under concurrent access.

For storage modelling, the system partitions data across Column Families (CFs): inode attributes and directory entries occupy separate CFs, each tuned with its own Bloom filter and compaction policy. Keys in the directory-entry CF are formed by concatenating the parent inode number and the child filename in big-endian order, so every directory entry corresponds to an edge in the directory tree. File operations thus reduce to edge operations—`create` inserts an edge, `unlink` removes one, and `rename`

deletes the old edge before inserting a new one—all at constant cost. By comparison, schemes that key by full path must rewrite every key in the subtree on rename, with cost scaling linearly in subtree size. Moreover, all child edges of the same parent are adjacent in key space, so directory traversal and single-entry lookups can be served by prefix scan and point query, respectively.

For write amplification, POSIX semantics require that `create` and `unlink` synchronously update the parent directory’s timestamps and link count. Under high concurrency the same parent repeatedly becomes a write hot spot, and staying with read-modify-write makes long lock waits hard to avoid. To address this, a delta update and lazy folding mechanism (DeltaOp) is introduced: each write appends a small delta record without touching the base inode value, converting in-place rewrites into append writes; each read merges the base value and outstanding deltas on the fly so that callers always observe up-to-date attributes; a background thread periodically scans accumulated deltas and folds them back into the base value once a threshold is reached, keeping read-path overhead bounded while eliminating write contention on the parent directory.

For concurrency safety, when multiple threads execute `create`, `rmdir`, or `rename` on the same directory, a race window exists between the check and the modification: `create` must confirm the absence of a duplicate before inserting, `rmdir` must confirm that the directory is empty before deleting, and `rename` involves multi-key atomic modifications across directories. RocksDB Pessimistic Concurrency Control (PCC) transactions lock the involved keys one by one, wrapping both checks and mutations inside a single atomic operation and thereby eliminating these races.

A full `pjdfstest` run in a distributed deployment (client and `MetadataServer` on separate nodes) executes all 398 cases. Of these, 167 depend on `mknod` and `mkfifo`, which this work deliberately leaves unimplemented; 1 case exceeds the `DataServer`’s current 64 MiB per-file development limit; a further 48 are skipped by the `pjdfstest` harness itself (e.g., `utimensat`, `remount`), unrelated to the metadata design. The remaining 182 cases constitute the in-scope set for this work and all pass—a 100%

pass rate within scope.

For performance, the test cluster consists of one 64-core server and up to 96 two-core clients, driven by mdtest. Under $N \in \{2, 8, 32\}$, RucksFS is compared head-to-head with NFS v4.2 and JuiceFS+TiKV. Against JuiceFS+TiKV, RucksFS is roughly $1.5\text{--}1.7\times$, $1.8\text{--}2.2\times$, and $1.2\times$ faster on `create`, `remove`, and `stat` respectively, reflecting two effects: a single-node direct-to-RocksDB design does not pay the cost of TiKV’s Raft and two-phase commit, and DeltaOp relieves write contention on shared parent directories. Against NFS v4.2, at $N = 2$ RucksFS is slightly slower because each `create`, with attribute cache disabled, now takes two extra lookup RPCs; at $N = 32$, NFS is bounded by ext4’s `i_rwsem`, and RucksFS overtakes it on `create` by roughly $4.6\times$.

At $N \geq 64$, neither NFS nor JuiceFS+TiKV sustains reliable numbers. In this range the comparison shifts to an internal one between the Delta and NoDelta builds of RucksFS, sweeping $np = N \times r$. Once np exceeds 96, NoDelta cannot continue to scale due to repeated transaction conflicts and its throughput stabilises around 12,000 ops/s, whereas Delta continues to scale to 38,000 ops/s, with `create` and `remove` ratios reaching $2.96\times$ and $2.63\times$ respectively at $np = 384$. Under easy mode, in which each process uses an independent parent directory, the two variants are indistinguishable, confirming that the gain originates from eliminating write conflicts on shared parent directories.

Keywords: File System, Metadata Management, Key-Value Storage, LSM-tree, Delta Update, Pessimistic Concurrency Control, FUSE

目 录

摘 要	I
Abstract	III
1 绪 论	1
1.1 课题背景、目的与意义	1
1.2 国内外研究现状	6
1.3 论文的主要内容与结构	14
2 相关技术基础	16
2.1 FUSE 用户态文件系统框架	16
2.2 RocksDB 存储引擎	20
2.3 POSIX 文件系统语义	22
2.4 本章小结	24
3 元数据组织与事务模型	26
3.1 设计目标	26
3.2 系统整体架构	27
3.3 元数据模型与存储布局	28
3.4 事务模型与元数据一致性	32
3.5 核心操作实现	34
3.6 本章小结	39
4 跨服务协调与系统实现	40
4.1 客户端路由与语义编排	40
4.2 DataServer 与原始磁盘后端	41
4.3 FUSE 可见语义与缓存策略	41
4.4 gRPC 接口与模块边界	42
4.5 错误处理与恢复机制	43
4.6 本章小结	44
5 测试与结果分析	46

华中科技大学毕业设计

5.1 实验目的与测量范围	46
5.2 实验组织	47
5.3 实验环境	48
5.4 POSIX 合规性评估	52
5.5 分布式性能对比	56
5.6 DeltaOp 增量机制的内部对比	62
5.7 本章小结	67
6 总结与展望	69
6.1 工作总结	69
6.2 不足与展望	70
致谢	71
参考文献	73

1 绪 论

文件系统的元数据管理（inode 属性的维护、目录项的组织以及文件操作的事务性保证）是高并发场景下吞吐量的主要瓶颈之一。传统 B+ 树引擎在面对每秒数万次的小文件创建时，就地修改带来的随机 I/O 和目录锁串行化会明显拖累整体吞吐。基于 LSM-tree 的键值存储用顺序写入代替随机写入，为该问题提供了一条可行的替代路线。

1.1 课题背景、目的与意义

1.1.1 研究背景

大规模模型训练、容器化微服务和云端数据湖等场景有一个共同特征：它们产生并消费海量小文件。一次分布式训练作业可能在 checkpoint 阶段写出数十亿个参数分片；Kubernetes 集群中的 Pod 在启动过程中需要创建和读取大量配置文件与临时卷。百度沧海存储团队在 Mantle^[1] 中给出了一组直观的数据：其云对象存储管理的文件对象数已达万亿量级，元数据请求规模每年仍有数倍增长。当文件数量攀升至这一量级时，系统的性能瓶颈通常不在数据传输带宽，而在元数据路径上。Ren 和 Gibson^[2] 的测量表明，stat、open、readdir 等元数据操作占文件系统全部 I/O 操作的 60%–80%。单个热点目录下的文件数一旦达到百万级，元数据查询与更新的延迟将直接限制系统吞吐的上限。

现有的本地文件系统（ext4、XFS）采用 B+ 树或其变体（如 ext4 的 H-tree）组织目录项和 inode 索引。为了说明这些传统方案在高频元数据写入下为什么难以扩展，下面以最简单的 `create("/mnt/dir/newfile")` 为例，分析该操作在 ext4 上所产生的磁盘写入。

这样一次文件创建操作在 ext4 内部共涉及 5 类磁盘区域、至少 6 次写入：

- (1) **Journal 事务起始**。ext4 在 ordered 模式下首先向日志区域写入一条事务描述符块（descriptor block），标记本次操作的开始。日志区域位于分区上的固定偏移量，与用户数据不相邻。
- (2) **inode bitmap**。从新文件所属的块组中查找空闲 inode 位——读取 inode 位图块（4 KiB），翻转对应比特位，写回。位图块的磁盘位置取决于块组编号，

通常与上一步的日志区域相距甚远。

- (3) **inode 表**。在 inode 表中初始化新分配的 inode 条目：写入 mode、uid、gid、时间戳等共 256 字节的属性数据。inode 表位于该块组的固定偏移处，又是一次不同位置的写入。
- (4) **目录数据块**。在父目录 /mnt/dir 的数据块中插入一条目录项 (dirent)。ext4 的 H-tree 是 B+ 树的哈希变体，插入时可能需要读取并修改叶子节点的数据块，若叶节点已满还要分裂出新块。目录数据块散布在父目录所属的块组中。
- (5) **父目录 inode**。更新父目录的 mtime (修改时间) 和 ctime (状态变更时间)，如果创建的是子目录还要递增 nlink (链接计数)。父目录 inode 与新文件 inode 可能不在同一个块组中。

最后，ext4 将上述所有修改的元数据块写入 journal 区域，追加一条 commit 块，调用 fsync 确保日志落盘，然后才将实际数据写回原位 (称为 checkpoint)。单次文件创建总共产生至少 6 次磁盘写入，分布在 inode bitmap、inode 表、目录数据块、父目录 inode 和 journal 等互不相邻的磁盘区域上——每一次都是随机 I/O。图 1-1 展示了这一过程。

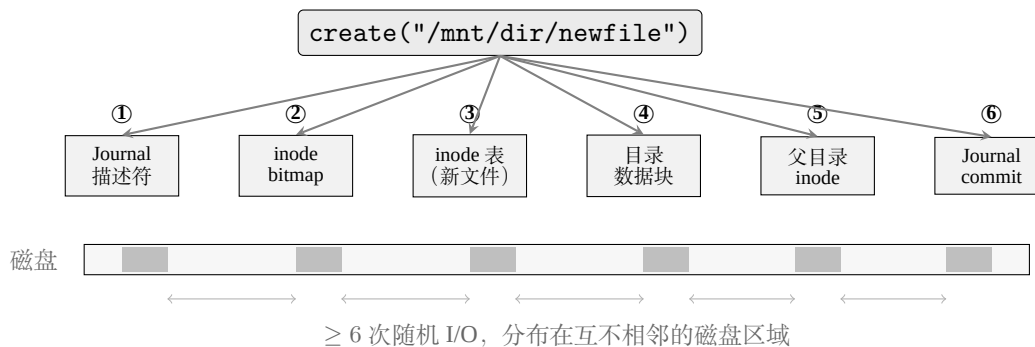


图 1-1 ext4 中一次 create 操作的磁盘写入路径。单次文件创建至少触及 6 个分散的磁盘区域，每次写入都是随机 I/O。

问题的根本在于粒度不匹配。一次文件创建真正需要写入的有效数据不超过 300 字节 (256 字节的 inode 加几十字节的目录项)，但 ext4 的操作粒度是 4 KiB 的磁盘页——即使只修改 256 字节的 inode，也需要读出整个 4 KiB 页、修改后再写回。Journal 模式下该页还会被写两次 (一次进 journal，一次落原位)。如果目录的 H-tree 恰好触发节点分裂，写入量还会再翻倍。单次操作的开销尚

可接受，但当每秒需要执行数万次 `create` 时，写放大的累积效应会明显拖累系统吞吐。

并发场景下情况更为严峻：`ext4` 对目录数据块的修改受目录级互斥锁 (`i_rwsem`) 保护，同一目录下的所有文件创建操作必须串行通过这把锁。即便底层存储是 NVMe SSD，能提供数十万 IOPS，单目录下的创建吞吐仍会被该锁压到远低于硬件能力的水平。

基于 LSM-tree (Log-Structured Merge-tree)^[3] 的键值 (KV) 存储为该问题提供了一条不同的技术路线。KV 存储将操作粒度从 4 KiB 的磁盘页细化到单条记录：一个目录项即一个键值对，写入时只做一次内存追加，不涉及 bitmap 翻转、journal 双写或 4 KiB 对齐的页填充。MemTable 充满之后一次性顺序刷盘，把原本分散的随机写归并为连续的顺序 I/O。TableFS^[2] 在 2013 年将全部元数据存入 LevelDB^[4] 并通过 FUSE 挂载，小文件创建吞吐达到同期 `ext4` 的 2-3 倍，率先验证了这条路线的可行性。此后 RocksDB^[5] 在 LevelDB 基础上加入了列族、事务和前缀布隆过滤器等特性；Apache Ozone^[6] 和 Facebook Tectonic^[7] 先后在 EB 级生产环境中使用 RocksDB 作为元数据引擎。该技术路线的整体可行性已基本清楚，尚未被完整回答的是：键编码、写放大控制和事务保障这几件事应当如何具体组织。

1.1.2 面临的问题和挑战

将 KV 引擎用于文件系统元数据管理，需要解决三个核心设计问题 (图 1-2)。

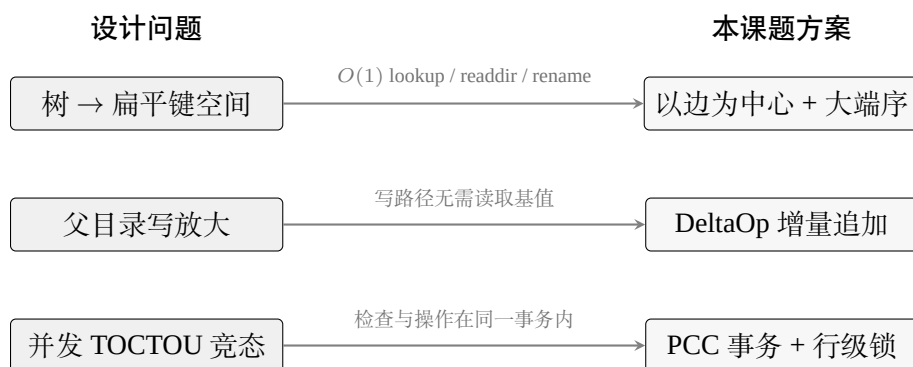


图 1-2 本课题面临的三个设计问题及对应解决方案。

问题一：树形目录到扁平键空间的映射。 KV 引擎看到的是字节串到字节串的扁平映射，而文件系统对外暴露的命名空间是一棵树。如何为树上每个对象

设计键，使得创建、查询、遍历、重命名都能高效完成，是首先需要想清楚的问题。

以 `create` 为例。`ext4` 中一次文件创建要写 6 个分散的磁盘区域（见前文 1.1.1 节）；在 KV 存储中，如果把每条目录项建模为一个键值对，`create` 就是插入一条记录，一次内存追加即可。但这一简化能否成立，取决于键的编码方式。

键的编码需要同时满足三项要求：(1) `create` 和 `unlink` 应为 $O(1)$ 的单条增删；(2) `readdir` 希望同一父目录下的子项在键空间中相邻，使得一次前缀扫描即可完成；(3) `lookup` 在拿到父 `inode` 号和文件名后能通过一次点查定位。若直接以全路径 (`/a/b/c`) 作为键，`create` 尚可，但一次 `rename` 需要递归改写整棵子树的所有键，开销与子树规模成正比。`TableFS`^[2] 以“父 `inode` + 文件名”拼接作为键，`create`、`lookup`、`rename` 均为 $O(1)$ ；但其 `inode` 号采用原生字节序存储，同一目录的条目在 LSM-tree 中未必物理相邻，前缀布隆过滤器也就无法发挥作用。因此键编码还需保持字节序与数值序一致 (`byte-order preserving`)，否则范围查询将给出错误结果。

问题二：父目录更新引发的写放大。 POSIX 语义^[8] 要求目录项发生变化时同步更新父目录的 `mtime` 和 `ctime`；仅在 `mkdir` 和 `rmdir` 这类子目录增删时才会进一步更新父目录的 `nlink`。在 KV 存储中，这意味着高频的 `create`、`unlink`、`mkdir`、`rmdir` 会反复修改同一条父目录 `inode` 记录。对于热点目录 (`/tmp`、CI 产物目录、模型训练的 `checkpoint` 目录)，若 1 秒内发生 1,000 次文件创建，父目录 `inode` 将被读写 1,000 次，其中大多数读到的值在写回前就已过期。在 RocksDB 中，这种对同一 `key` 的密集 `Get+Put` 会让 `MemTable` 更快写满、L0 文件更快堆积，并引发更多 `compaction`，使写放大连锁加剧。

问题三：并发目录操作的正确性。 多个线程同时操作同一目录时，“检查条件”和“执行修改”之间存在一个时间窗口，即 TOCTOU (`Time-of-Check-to-Time-of-Use`) 竞态。例如线程 A 执行 `rename` 时已确认目标路径不存在，但在其写入新目录项之前，线程 B 已在同一位置创建了同名文件，两者发生碰撞会导致目录项冲突或数据丢失。内核文件系统由 VFS 层的 `inode` 互斥锁将目录操作串行化；而在 KV 引擎之上没有这一层保护，只能依赖存储层自身的事务机制。锁粒度的选择——锁整个目录还是仅锁涉及的 `key`——又直接关系到并发度

与实现复杂度之间的取舍。

1.1.3 课题目的与意义

本课题的目标是：设计并实现一个基于 RocksDB 的用户态 FUSE 文件系统 RucksFS，针对上述三个问题给出可落地的方案，并通过实验验证方案是否达到预期。

技术选型。 元数据引擎选用 RocksDB，是因为它的几项特性恰好对应上述三个问题：Column Family 可以把 inode 属性、目录项、增量记录放在相互独立的键空间中各自配置参数；前缀迭代器和前缀 Bloom 过滤器能够支撑目录前缀扫描；悲观事务（TransactionDB）提供了行级锁和冲突检测，可用于避免 TOCTOU 竞态。对外接口选择 FUSE 出于工程层面的考虑——用户态开发迭代快、调试方便，与 JuiceFS^[9]、CubeFS^[10] 的接入方式一致，而且 FUSE 的操作接口与内核 VFS 基本对应，未来若要迁移到内核态改动量不至于过大。实现语言采用 Rust^[11]：一方面文件系统这类基础设施对内存安全要求较高，Rust 在编译期即可排除大部分相关缺陷；另一方面它的 async/await 生态（tokio）与 FUSE 的异步处理模式契合度较高。

解决思路。 对应上述三个问题，本课题分别给出三项设计。键编码采用“以边为中心”的思路，将父 inode 号与子文件名按大端序拼接作为目录项键，使文件操作对应于目录树中一条边的常数时间增删。父目录写放大的问题通过增量更新（DeltaOp）解决——写入时只追加一条变化量，不修改基础 inode 值，由后台线程定期将累积的增量折叠回基值。并发正确性由 RocksDB 悲观事务保证，将检查与修改封装在同一事务内；按 inode ID 排序加锁以预防死锁。三项设计的细节将在第 3 章展开。

课题意义。 从技术角度看，TableFS 时代的 KV 元数据系统普遍缺少事务能力，IndexFS 则将重心放在跨节点扩展上；RocksDB 自 6.0 版起提供了成熟的悲观事务接口，但目前尚未看到系统性地将其应用于 POSIX 文件操作元数据层的公开工作。本课题尝试填补这一空白的一部分，给出一个可复现的原型和对照实验。从工程角度看，RucksFS 的 Rust 实现与三段式分布式部署形式可以作为后续工作的基础——例如后续加入元数据分片、副本容错或智能预取时，这套

骨架无需重构。

预期交付物。（1）按 FUSE 客户端、MetadataServer 和 DataServer 解耦的 RucksFS 原型，支持 gRPC 远程通信与分离部署；（2）pjdfstest POSIX 合规性测试结果；（3）在同一硬件条件下与 JuiceFS、NFS 等系统的元数据吞吐量对比数据。

1.2 国内外研究现状

用 KV 引擎管理文件系统元数据的研究始于 2013 年，至今已积累了从单机原型到 EB 级生产系统的丰富实践。

1.2.1 基于 KV 存储的元数据管理研究

TableFS^[2] 是这一方向最早的系统性工作。2013 年，Carnegie Mellon University 的 Ren 和 Gibson 将 ext4 的目录项和 inode 信息整体存入 LevelDB，并通过 FUSE 挂载为标准文件系统。TableFS 的设计思路直截了当：以（父 inode 号，文件名）的拼接作为 LevelDB 的键，以序列化后的 `struct stat` 加上可选的内联数据作为值。在小文件（< 4 KB）场景下，TableFS 的 `create` 吞吐量是同期 ext4 的 2–3 倍，这一结果首次验证了 KV 存储用于元数据管理的可行性。但 TableFS 也暴露了若干缺陷：（1）`readdir` 操作需要扫描 LevelDB 的全部键空间并过滤出指定父目录的条目——当文件系统中的文件总数增长到百万级别时，`readdir` 的延迟急剧上升；（2）LevelDB 本身不提供任何事务语义，多线程写入时的正确性完全依赖外部加锁；（3）键编码方案中父目录 inode 号使用原生字节序存储，导致同一目录下的条目在 LSM-tree 中不一定连续排列，无法利用前缀迭代器或前缀布隆过滤器加速查找。此外，TableFS 没有考虑父目录 inode 更新带来的写放大问题。

同一研究团队在次年推出了 IndexFS^[12]，将 TableFS 的思路扩展到分布式环境。IndexFS 在每个元数据节点上运行一个 LevelDB 实例，目录项通过 GIGA+ 动态哈希分区算法分布到不同节点。其关键贡献是引入了列式目录布局（columnar directory layout）：将 `readdir` 所需的信息（文件名列表）与 `getattr` 所需的信息（inode 属性）分开存储在不同的 SSTable 列中，使得目录遍历只需读取文件名列，跳过体积更大的 inode 属性数据。IndexFS 在 128 节点的集群上实现了每秒超过 100 万次 `create` 操作，获得了 SC'14 的最佳论文奖。不过 IndexFS 的部

署要求相当苛刻：每个节点需要独立维护一个 LevelDB 实例和一套目录缓存状态机，还需要一个独立的 GIGA+ 组件来处理跨节点的目录分裂。对于中小规模的使用场景，这样的架构存在过度设计之嫌。

LocoFS^[13] 在 2017 年观察到文件访问的时间局部性和空间局部性，设计了一种松耦合的元数据分布策略，按哈希将热点目录的元数据分配到固定服务器。Xu 等人^[14] 在 2014 年提出了面向 EB 级规模的子树粒度元数据划分方案。这些工作侧重多节点元数据划分与扩展策略，与本课题关注的“分布式访问路径下，如何围绕 RocksDB 组织 POSIX 元数据操作”这一问题并不相同。

百度沧海·存储团队在 SOSP'25 上发表的 Mantle^[1] 是 KV 元数据管理这一技术路线的最新进展。Mantle 为云对象存储提供层级命名空间支持，底层持久化引擎 TafDB 采用三个列族组织元数据：AccessCF 以（父 inode 号 + 文件名）为键存储目录项，AttrCF 以 inode 号为键存储属性，DeltaCF 以（inode 号 + 时间戳）为键追加增量更新记录。其中 Delta Record 机制与本课题的设计动机直接相关：当多个客户端并发在同一目录下创建文件时，父目录的子项计数和 mtime 不再以读取-修改-写回的方式更新，而是各自追加一条增量记录，由后台线程异步将增量合并到 AttrCF 中的基准值。这种设计完全消除了热点目录属性的写竞争。

Mantle 还在 TafDB 之上叠加了 IndexNode 内存缓存层，以加速长路径解析和权限检查，在百度内部的 EB 级数据湖上实现了相比旧方案 115 倍的吞吐量提升。不过 Mantle 依赖的 TafDB 是百度自研的类 Spanner 分布式事务数据库，IndexNode 与 TafDB 之间的缓存一致性协议也相当复杂——整套系统并不开源。本课题借鉴了 Mantle 的增量更新思路，但将其落到“Client + MetadataServer + DataServer”三段式分布式路径上：元数据仍由 MetadataServer 上的 RocksDB 统一维护，同时以 DeltaOp + 懒折叠降低热点目录的写竞争，用较低的系统复杂度复现其核心收益。

1.2.2 云原生文件系统中的元数据方案

在 KV 元数据研究逐步成熟的同时，工业界的分布式文件系统也在快速演进。本课题选择 JuiceFS 和 CubeFS 作为实验对比对象，这里简要介绍二者的元数据方案以及其他几个代表性系统的技术路线，为后续对比提供背景。

早期的分布式文件系统以集中式元数据管理为主：GFS^[15] 和 HDFS^[16] 将全部元数据驻留在单节点内存中，Ceph^[17] 用动态子树划分将目录树分配到多个元数据服务器上。这些系统在 PB 级规模上运转良好，但元数据的可扩展性始终受到内存容量或子树迁移开销的制约。分布式 KV 存储在可用性优先的设计思路（如 Amazon Dynamo^[18]）逐渐成为上层文件系统的新型后端选择之后，Apache Ozone^[6] 和 Facebook Tectonic^[7] 在 2020 年前后将元数据引擎替换为 RocksDB，分别管理了 EB 级的数据，但两者都不提供完整的 POSIX 接口——Ozone 面向 S3 和 Hadoop API，Tectonic 面向 Facebook 内部的对象存储协议。

JuiceFS^[9] 是国内使用较广的开源 POSIX 文件系统。它的架构将元数据和数据完全分离：元数据存储于 Redis、TiKV 或 MySQL 等外部引擎中，数据块则写入 S3 兼容的对象存储。这种设计的优势在于部署灵活——用户可以根据自身条件选择最合适的后端组合。劣势则在于性能和可靠性完全取决于外部组件：Redis 的单线程模型在元数据操作超过每秒数万次时会成为瓶颈；TiKV 虽然吞吐量更高，却引入了 Raft 共识协议带来的额外延迟。JuiceFS 官方基准测试显示，Redis 后端在单客户端下的文件创建吞吐量约为 1,410 ops/s，而 MySQL 后端仅约 350 ops/s。JuiceFS 的 FUSE 客户端使用 Go 语言实现，Go 运行时的垃圾回收（GC）暂停在极端负载下曾被部分用户报告为延迟抖动的来源之一。

CubeFS^[10]（最初名为 CFS）于 2019 年在 SIGMOD 上发表，后捐赠给 CNCF 并于 2024 年成为毕业项目。与 JuiceFS 依赖外部数据库不同，CubeFS 自带元数据引擎：每个元数据分区（MetaPartition）使用内存中的 B-tree 维护目录结构，多副本之间通过 Multi-Raft 协议保持一致性。CubeFS 的读路径延迟很低（内存 B-tree 的查找延迟在微秒级），但写路径受 Raft 日志同步所限——每次写操作至少需要等待多数副本确认，典型延迟在毫秒级。内存中的 B-tree 也限制了单节点可管理的元数据量：按每个 inode 约 300 字节内存估算，十亿文件需要约 280 GB 内存。

上述系统的元数据管理方案按技术路线可归为三类：内存驻留型（GFS、HDFS、CubeFS）、外部数据库型（JuiceFS）、嵌入式 KV 型（Ozone、Tectonic）。三类方案各有侧重，一个共同趋势是围绕分布式环境组织元数据服务：要么把一致性和事务交给外部数据库，要么把复制、分片和故障恢复做进文件系统自

身。这也带来一个更具体的问题：在已经采用分布式部署的前提下，元数据引擎本身该如何围绕文件操作做定制，而不是完全受制于通用数据库或共识框架的语义与开销。

1.2.3 元数据引擎的深度优化研究

与单纯追求分片扩展不同，另一条研究路线更关注元数据引擎本身的执行效率，即先缩短单次操作的关键路径，再讨论系统如何扩展。SingularFS^[19] 的成果表明，这一路线的性能上限远高于通常的直觉判断。

SingularFS 在 ATC'23 上展示了一个激进的设计选择：仅用单个元数据服务器支撑十亿级文件规模的分布式文件系统。其核心优化包括三个方面：(1) 无日志元数据操作 (log-free metadata operations)，通过精心设计的持久化顺序消除崩溃一致性日志的开销；(2) 层次化并发控制，对目录树的不同层级使用不同粒度的锁，避免粗粒度锁的串行化瓶颈；(3) NUMA 感知的 inode 分区，将 inode 地址空间按 NUMA 节点划分，减少跨节点内存访问。在配备 RDMA 网络和持久内存的硬件环境下，SingularFS 的单节点元数据吞吐量远超传统多节点方案。这一结果对本课题的启发不在于“把系统做成单机”，而在于即便系统整体以分布式方式部署，元数据服务内部仍值得围绕关键路径做深度优化。RucksFS 当前正是沿着这条思路，在 MetadataServer 的 RocksDB 事务、增量更新和键编码上做针对性设计，而不是把全部语义都外包给通用分布式 KV。

Wang 等人^[20] 在 EuroSys'23 上发表的 CFS 对元数据服务中临界区 (critical section) 的范围做了精细化分析。传统方案在处理 rename 等涉及多个目录的操作时，往往采用粗粒度的目录级锁，导致不相关的操作被串行化。CFS 通过形式化方法识别出各个 POSIX 操作中真正需要互斥的 key 集合，将临界区缩减到最小。实验表明，这种优化将 rename 密集型工作负载的吞吐量提升了 3.2×。本课题的 PCC 事务设计借鉴了类似思路——只对实际涉及的 key 加排他锁，而非锁定整个目录，从而在保证正确性的同时保留并发度。

Yang 等人^[21] 在 2020 年从实用角度分析了大规模目录中逐个执行 stat 和 open 效率低下的问题，设计了将 readdir+stat 复合操作在内核内合并执行的批量接口，减少了用户态/内核态切换次数。这一批量化思路直接影响了 RucksFS

的 `readdirplus` 实现——在一次 FUSE 调用中同时返回目录项和完整的 inode 属性，避免后续逐个 `getattr` 的往返开销。

操作系统层面也有相关的优化工作。FetchBPF^[22] 发表于 ATC'24，利用 eBPF 在 Linux 内核中实现可定制的文件访问预取策略。用户编写的 eBPF 程序由内核在 I/O 路径中直接执行，避免了用户态预取方案的上下文切换开销，在顺序读工作负载上接近理论带宽。同期的 FBMM^[23] 则展示了将 Linux VFS 接口用于非传统目的的可能性——将内存管理器实现为文件系统，利用 VFS 的可扩展性在不修改内核核心代码的情况下引入新策略。这种“文件系统即抽象层”的理念与本课题使用 FUSE 将 KV 存储暴露为 POSIX 接口的思路有相似之处。FetchBPF 所展示的 eBPF 预取能力也为 RucksFS 未来的优化方向提供了参考——在 FUSE 的 `/dev/fuse` 通道上通过 eBPF 预取即将被访问的元数据，有可能降低用户态守护进程的响应延迟。

MetaHive^[24] 在 2024 年针对 KV 存储引擎在元数据场景下的缓存效率问题展开研究。通用 KV 存储的缓存替换策略（如 LRU）未考虑文件系统的访问局部性——同一目录下的 inode 被连续访问的概率远高于随机 key。MetaHive 据此设计了元数据感知的缓存分区方案。这一发现与 RucksFS 的 InodeFoldedCache 设计动机一致：系统在应用层维护了专门的 LRU 缓存来存储折叠后的 inode 值，而非依赖 RocksDB 的通用块缓存。沿着“为文件系统元数据定制执行路径”这一方向，FUSEE^[25] 将 KV 元数据服务下沉到内存解耦架构上；Dong 等人^[26] 在 ICCD 2023 上讨论了完整路径索引下的低延迟元数据服务设计；SwitchFS^[27] 则在交换机侧协调元数据异步更新。这些工作的方向与本课题并不完全重合，但都表明元数据引擎本身仍有较大的优化空间。

在国内，龚拓宇^[28] 在 2020 年的清华大学硕士论文中结合 RDMA 技术与 RocksDB 设计了分布式元数据服务原型 RocksMeta，在单节点上实现了超过 10 万 IOPS 的元数据操作吞吐量。其键编码方案与本课题有相似之处，但侧重网络传输优化（RDMA 旁路内核协议栈），而本课题侧重存储层的写放大优化。范立衡等人^[29] 较早探索了键值存储用于元数据集副本一致性的管理策略。吴雨飞等人^[30] 在 2025 年针对基于 TiKV 的分布式文件系统元数据缓存一致性问题，提出了并发广播与惰性删除相结合的轻量级维护机制。

表 1-1 按“对本课题架构选择的直接影响程度”筛选并汇总了五项最具参考意义的相关工作。其余工作——JuiceFS 作为实验主对比对象，CubeFS 作为技术路线参照，CFS 作为细粒度临界区分析的思想来源，Yang 等人的批量接口对应 `readdirplus` 实现，FetchBPF 与 FBMM 作为面向操作系统层的周边借鉴——均已在正文中展开讨论，此处不再重复罗列，以保持表格清晰可读。

表 1-1 五项最具参考意义的 KV 元数据相关工作

工作	类别	对本课题的参考
TableFS ^[2]	KV 元数据	验证了 KV 存储用于元数据的可行性；暴露键编码、 <code>readdir</code> 效率和事务缺失三个问题，本课题逐一改进
IndexFS ^[12]	KV 元数据	列式目录布局启发了 <code>readdirplus</code> 设计；其目录分片思路为后续元数据横向扩展提供参考
Mantle ^[1]	KV 元数据	三列族设计和 Delta Record 机制直接启发本课题的多列族架构和 DeltaOp 懒折叠方案；工业级验证增强可信度
SingularFS ^[19]	单机 MDS	证明元数据关键路径的深度优化本身可以显著抬高分布式文件系统的性能上限
CFS ^[20]	并发控制	细粒度临界区分析方法影响本课题的 PCC 事务设计——只锁涉及的 key 而非整个目录

上述工作各自解决了一部分问题；但若要把这些思路组合到一个“Client + MetadataServer + DataServer + FUSE + RocksDB + 完整 POSIX”的最小可用系统中，仍有若干问题需要回答。下一节先把这些空白点明，再给出 RucksFS 与它们的具体定位对比。

1.2.4 现有研究不足与本课题定位

综合上述工作，有几个问题并未被完整回答。这些问题并非学术界彻底忽视的盲点，更多是在某些系统中部分解决过，但尚未在“分布式访问路径 + POSIX + 嵌入式 KV”这一完整组合下被系统处理。具体有三点。

一是事务和并发控制这一环较为薄弱。TableFS^[2]所用的 LevelDB 没有事务语义，多进程写入只能在上层依靠粗粒度锁保护——单用户基准测试下问题不明显，但在 make 并行构建、rsync 同步、并发日志写入等真实负载下，竞态就会暴露出来。IndexFS^[12]引入了 WriteBatch 来保证单次操作的原子性，但 rename 同时涉及源目录和目标目录的多条记录，WriteBatch 并不覆盖跨操作的一致性约束——跨目录 rename 在 IndexFS 中仍需要上层自行加锁。RocksDB 自 6.0 版起已经提供了成熟的悲观事务 (PessimisticTransactionDB)，支持行级锁、GetForUpdate 和冲突检测；配合大端键编码的前缀局部性，还能把冲突范围限制在单个父目录内。问题在于，将其系统性地应用到文件系统元数据操作上的公开工作目前尚未见到。

二是父目录 inode 的写放大问题长期被忽略。TableFS、JuiceFS、CubeFS 处理目录变更时基本采用读-改-写方式：create 和 unlink 需要更新父目录的 mtime 和 ctime，mkdir 和 rmdir 还会进一步更新父目录的 nlink。这些字段每次都被完整读出、修改后再写回。在热点目录下（例如存放了数十万文件的临时目录），这种模式会把所有写入串到同一条父 inode 记录上，LSM-Tree 擅长的顺序追加特性反而无法发挥。Mantle^[1]的 Delta Record 是目前唯一在生产环境中系统性解决该问题的方案，但它依赖的 TafDB 分布式事务数据库和 IndexNode 缓存一致性协议整体较为复杂，难以直接引入本课题这种轻量的三段式架构。如何在 MetadataServer 本地的 RocksDB 上以较低复杂度实现类似的增量更新语义、同时保证 POSIX 的原子性和读一致性，目前没有现成的答案。

三是现有系统普遍偏向通用分布式可扩展性，对专用元数据服务器本身的执行路径关注不足。IndexFS、CubeFS、Tectonic 普遍引入 Raft 共识、一致性哈希、分区迁移等重型机制——在 EB 级规模下这些是必要的基础设施，但同时也会使原本可以在专用元数据服务器内部快速完成的简单目录操作，退化为较重

的事务流程。SingularFS 的实验数据表明，元数据引擎本身经过深度优化后可达到的吞吐上限相当高；只是学术界更倾向于发表大规模分布式系统，“分布式部署 + 专用元数据引擎 + POSIX 合规”这一组合下的实现细节反而少有公开、完整的原型。毕业设计这类既需要真实分布式访问路径、又要控制系统复杂度的工作，正好对应上述空白。

把上述三点合起来看，本课题的定位就比较清晰：设计并实现一个基于 RocksDB 的分布式文件系统原型 RucksFS，由 FUSE Client、MetadataServer、DataServer 三部分组成，通过 gRPC 连接客户端与服务端；元数据侧以 RocksDB 为基础，采用多列族、悲观事务和前缀迭代器等特性，配合一套适合文件系统访问模式的键编码和增量更新机制。与已有工作的主要区别集中在三处：（1）通过 PCC 事务为所有目录变更操作统一提供原子性保证，而非依赖外部锁或仅使用 WriteBatch；（2）借鉴 Mantle 的 Delta Record 思路，使用 DeltaOp + 懒折叠解决父目录 inode 写放大，但相关逻辑均在 MetadataServer 本地 RocksDB 内完成，不引入外部通用分布式事务数据库和复杂的缓存一致性协议；（3）以 Rust 实现，基于 FUSE 异步运行时和 gRPC 通信构建完整的分布式访问路径，在不修改内核、不依赖特殊硬件的前提下验证该架构的语义和性能表现。表 1-2 从 KV 引擎、事务支持、增量更新、用户态接口这四个维度，将本课题与相关系统并列对比。

表 1-2 本课题与相关系统的定位对比

系统	KV 引擎	事务支持	增量更新	用户态接口
TableFS ^[2]	LevelDB	无	无	FUSE
IndexFS ^[12]	LevelDB	WriteBatch	无	自定义库
Mantle ^[1]	TafDB	分布式事务	Delta Record	对象存储 API
JuiceFS ^[9]	多种后端	依赖后端	无	FUSE
CubeFS ^[10]	内存 B-tree	Raft	无	FUSE
RucksFS（本课题）	RocksDB	PCC 事务	DeltaOp	FUSE

1.3 论文的主要内容与结构

1.3.1 主要工作

本课题围绕“基于 KV 存储的文件系统元数据管理”这一目标，主要完成了以下工作：

(1) 设计并实现了基于 RocksDB 多列族的元数据存储方案。不同类型的元数据（inode 属性、目录项、增量记录）分别放在独立列族中，各自配置压缩策略和缓存参数。目录项列族的键采用“以边为中心”的编码：父 inode 号和子文件名按大端序拼接作为键，每条目录项对应目录树中一条父子边。文件操作由此转化为对边的增删——`create` 插入一条、`unlink` 删除一条、`rename` 删旧插新——均为常数时间。大端序编码使字节序与数值序一致，同一父目录下的子项在键空间中自然相邻，`readdir` 通过前缀扫描、`lookup` 通过点查即可完成。

(2) 引入了增量更新与懒折叠（`lazy folding`）机制。文件创建和删除不直接修改父目录 inode，而是以 `DeltaOp` 形式追加一条增量记录；读路径在 `getattr` 或 `readdir` 时将基值与增量在线合并后返回。后台线程在增量数量超过 32 条时将其折叠回基值。由此在高并发写入下，KV 层面的“读-改-写”次数明显减少。

(3) 实现了相对完整的 POSIX 文件操作集合——`create`、`mkdir`、`unlink`、`rmdir`、`rename`、`readdir`、`setattr`、`read`、`write` 等。所有变更操作通过 RocksDB 悲观事务完成加锁和冲突检测，以原子提交保证多键更新的一致性；事务冲突时服务端先自动重试最多 3 次，仍失败后在 FUSE 层映射为 `EAGAIN` 返回。

(4) FUSE 客户端基于 Rust 的 `fuse3` 库（0.8 版）与 `tokio` 异步运行时实现，按异步模型处理请求。挂载时启用 `default_permissions` 和 `allow_other`，将常规权限检查交由内核 VFS 处理。

(5) 建立了多层次的测试体系：约 192 个单元测试覆盖核心逻辑，`pjdfstest` 套件用于验证 POSIX 兼容性，另有一套自定义的元数据基准（`create/stat/unlink/readdir` 吞吐和延迟）用于跨系统对比；在同一硬件条件下与 JuiceFS、NFS 共同进行了评测。

上述实现的完整源代码、测试编排脚本、本文所用的原始测量数据以及本论文的 LaTeX 源码均公开在 <https://github.com/csjpgg/rucksfs>，按第 5 章的

参数即可复现。

1.3.2 论文结构

本论文共分为五章，各章安排如下：

第一章绪论。交代研究背景，分析 KV 存储用于文件系统元数据时需要面对的技术挑战，梳理国内外相关工作，最后给出本课题的目标与定位。

第二章相关技术基础。介绍 FUSE 的工作原理和请求处理流程，RocksDB 的 LSM-tree 架构、列族与事务机制，POSIX 文件系统语义的几项关键约束，以及悲观并发控制（PCC）的基本概念。

第三章系统设计与实现。展开 RucksFS 的整体架构（Client、MetadataServer、DataServer 与 7 个 crate 组成的 workspace 模块划分），元数据键编码方案，增量更新与懒折叠机制，PCC 事务封装与冲突重试策略，以及各 POSIX 操作的具体实现。

第四章测试与结果分析。先交代测试环境和具体的实验方法，然后给出 RucksFS 与 JuiceFS、NFS 在元数据操作吞吐、延迟、POSIX 合规性上的对比数据，并对结果进行分析。

第五章总结与展望。回顾主要成果，讨论当前设计的局限，给出元数据分片、副本容错、RPC 路径优化和智能预取等方向的后续改进思路。

2 相关技术基础

RucksFS 的设计建立在三组基础技术之上：FUSE 负责将 Linux VFS 请求交给用户态客户端，RocksDB 提供适合元数据工作负载的持久化引擎和事务能力，POSIX 语义则规定了文件系统操作应当呈现的行为。本章只介绍与后续设计直接相关的部分，不对这三个主题作全面铺陈。

2.1 FUSE 用户态文件系统框架

ext4、XFS、Btrfs 这类传统文件系统运行在内核态，调用路径短，但开发、调试和故障定位都需要直接面对内核环境。FUSE (Filesystem in Userspace)^[31] 采用了不同的思路：内核只保留请求转发、进程阻塞/唤醒、权限协作等基础机制，具体的文件系统逻辑放到用户空间实现。从 Linux 2.6.14 起 FUSE 进入主线内核，随后逐渐成为用户态文件系统的事实标准。JuiceFS^[9]、CubeFS^[10]、GlusterFS、SSHFS 均通过 FUSE 对外提供 POSIX 接口。

本课题选择 FUSE 不是为了规避内核态开发的难度，而是为了获得一个清晰的系统边界。Linux VFS 继续负责系统调用入口与权限框架，用户态客户端只负责协议解析、请求路由，以及与元数据服务、数据服务交互。这种分工与后续三段式分布式架构相匹配。

2.1.1 工作机制

FUSE 在 RucksFS 中的请求通路可以抽象为表 2-1 中的 6 个阶段。表中重点在各段的边界与职责，而非具体函数名。

当应用对挂载点执行 `open()`、`stat()`、`mkdir()` 等系统调用时，请求首先进入 VFS。若目标路径位于 FUSE 挂载点下，VFS 不会继续调用 ext4 之类的内核文件系统实现，而是把请求转交给 FUSE 内核模块。该模块为请求分配请求 ID，封装操作码与参数，并将请求写入 `/dev/fuse`，同时让发起系统调用的线程阻塞等待。

用户态守护进程在挂载时已经打开 `/dev/fuse`，随后持续从该设备读取请求帧。守护进程根据 opcode 识别请求类型，再把请求派发到开发者实现的回调，例如把 LOOKUP 派发给 `lookup()`，把 CREATE 派发给 `create()`。回调完成后，守

表 2-1 FUSE 请求的分层执行路径

阶段	位置	动作
1	应用进程	调用 <code>open()</code> 、 <code>stat()</code> 、 <code>mkdir()</code> 等 POSIX 接口
2	Linux VFS	识别目标路径属于 FUSE 挂载点，并把请求转交 FUSE 内核模块
3	FUSE 内核模块	为请求分配 ID，封装 <code>opcode</code> 与参数，写入 <code>/dev/fuse</code> ，并阻塞原线程
4	FUSE 客户端	从 <code>/dev/fuse</code> 读取请求，解析协议头并派发到对应回调
5	RucksFS 用户态逻辑	通过 <code>VfsCore</code> 调用 <code>MetadataServer</code> 或 <code>DataSeter</code> ，得到处理结果
6	内核返回路径	守护进程写回响应，内核按请求 ID 唤醒原线程并返回系统调用结果

护进程把结果编码为响应帧并写回 `/dev/fuse`；内核再据请求 ID 唤醒原先阻塞的线程，将结果接回系统调用返回路径。

从系统结构上看，FUSE 把“POSIX 请求如何进入文件系统”与“文件系统如何实现这些请求”这两件事清晰地分开。内核负责入口与调度，用户空间负责语义与存储实现，二者之间唯一共享的交互媒介就是 `/dev/fuse`。

2.1.2 回调模型与异步实现

FUSE 向开发者暴露的是一组与 VFS 操作一一对应的回调接口。表 2-2 列出了本文直接涉及的核心回调。它们与 POSIX 操作基本一一对应：`lookup` 用于路径解析，`getattr` 对应 `stat()`，`create` 对应 `open(O_CREAT)`，`rename` 对应目录项移动，`readdir` 对应目录遍历。

在实现机制上，`fuse3` 系列 Rust 绑定（如 `fuser`^[32]）的异步模型可以概括为“读请求帧、生成 `Future`、等待完成后写回响应”三个步骤。运行时从 `/dev/fuse` 读到请求后，并不为每个请求单独分配一个阻塞线程，而是将请求派发到对应

表 2-2 FUSE 核心回调接口（本文涉及部分）

回调	对应 VFS 操作	语义
lookup	路径解析	给定父 inode 和文件名，返回子 inode 的属性
getattr	stat()	返回 inode 的元数据（mode、size、mtime 等）
setattr	chmod/chown/utimes/truncate	修改 inode 属性
create	open(O_CREAT)	在父目录下创建文件并返回文件句柄
open	open()	为已有文件分配 file handle
release	close()	释放 file handle 及相关资源
mkdir	mkdir()	在父目录下创建子目录
unlink	unlink()	从父目录中删除文件的目录项
rmdir	rmdir()	删除空目录
rename	rename()	将目录项从源位置移动到目标位置
readdir	getdents()	列出目录下的所有条目
read	read()	读取文件数据
write	write()	写入文件数据
fsync	fsync()	将文件数据与元数据刷盘

的异步回调，由执行器推进这些 Future。任何回调在中途进入等待状态（例如在等待元数据 RPC、数据 RPC 或本地异步 I/O）时，执行器可以先将其挂起，转而处理其他已就绪的请求。

这种做法的关键不在于“把同步函数改写为异步语法”，而在于把等待的开销从内核工作线程中移出。对分布式文件系统的客户端而言，FUSE 回调之后往往伴随一次或多次网络往返；若仍采用“一个请求一个线程”的阻塞模型，线程数会随并发请求数和网络延迟一起增长。异步模型把并发控制交由事件循环和任务调度完成，在有限线程数下即可维持大量未完成的请求。

对 RucksFS 这种客户端/服务端分离的架构而言，fuse3 的异步接口较为合适。FUSE 层负责收发请求，真正耗时的是用户态客户端与 MetadataServer、DataServer 之间的交互。该路径做成异步后，在等待远端返回期间可以继续推进其他 lookup、getattr、readdir，对元数据密集的并发负载较为友好。

2.1.3 性能开销特征

FUSE 的主要代价来自跨内核态与用户态边界的固定成本。对 ext4 这类内核态文件系统，一次 stat() 基本在内核内完成；而对 FUSE 文件系统，同一次 stat() 至少需要走完“应用线程进入内核、请求写入 /dev/fuse、守护进程读取请求、守护进程写回响应、内核唤醒原线程”这一整套。与内核态相比，多出的开销主要有两类：上下文切换，以及请求/响应两帧在 /dev/fuse 上的拷贝。

Vangoor 等人^[33]进行了系统性测试：在 lookup、getattr 这类元数据操作上，FUSE 文件系统的吞吐较内核态文件系统通常下降 20%–60%；顺序大文件读写场景下，下降幅度一般只有 5%–20%。原因可以理解为：元数据操作本身执行时间短，固定的边界开销占比较高；一旦磁盘或网络 I/O 成为主导，固定成本就会被摊薄。

对 RucksFS 这种分布式原型而言，FUSE 并不是主要瓶颈。真正决定整体性能的是客户端到 MetadataServer 的 gRPC 网络路径：每次文件操作至少包含一次跨节点 RPC 往返，其延迟远大于 FUSE 自身的内核–用户态边界开销。因此 FUSE 引入的是一个固定的常数开销，而不是会随规模劣化的瓶颈。

2.2 RocksDB 存储引擎

RocksDB^[5] 是 Meta 在 LevelDB^[4] 基础上发展出的嵌入式键值存储引擎, 针对 SSD 场景和高写入负载做了较多工程优化, 已被 TiKV、MyRocks、CockroachDB 等上层系统广泛用作持久化后端。对“小值、高频、随机写密集”的文件系统元数据负载而言, RocksDB 的 LSM-tree、列族、前缀布隆过滤器和事务接口都是较为合适的工具。

2.2.1 LSM-tree 数据结构

LSM-tree (Log-Structured Merge-tree) 最早由 O’Neil 等人提出^[3], 其核心思想是把随机写换成顺序写: 新写入先进入内存中的 MemTable, 同时追加一条 WAL; MemTable 写满后作为一整块有序 SST 文件刷盘; 后台 Compaction 线程按键范围合并重写各层 SST, 以维持读性能和空间利用率。这一思路在更早的日志结构文件系统^[34] 中已有雏形, 之后被 Bigtable^[35]、LevelDB^[4] 等工业系统进一步发展成为面向 SSD 的高写入吞吐存储引擎。

对文件系统元数据而言, 这一结构有两点直接影响。其一, 单条目录项或 inode 属性的更新可以先在 MemTable 中完成一次内存追加, 而不像 B 树那样立即修改磁盘页。其二, 后台合并阶段会引入额外的写放大, 因此键的设计和值的大小需要保持克制。WiscKey^[36] 的做法是把 value 与 key 分离存储, 进一步降低 Compaction 时的数据搬移成本; RucksFS 则沿另一条路径——目录项和 inode value 都尽量保持小尺寸 (分别为 12 字节和 57 字节), 文件数据完全绕开 RocksDB, 避免大 value 对 Compaction 的影响。

RocksDB 的读取路径方向相反: 先查活跃 MemTable, 再查不可变 MemTable, 然后自 Level 0 开始逐层搜索 SST 文件。为了降低点查代价, SST 文件内部维护索引块和 Bloom 过滤器; 为了减少磁盘访问, 常用数据块会进入块缓存。这也是文件系统元数据需要强调键的前缀局部性和点查友好性的原因: 否则每次读取都会放大到多层 SST, 产生明显的读放大。

2.2.2 Column Family 机制

Column Family (列族) 是 RocksDB 较为实用的一项特性。它允许在同一个数据库实例内维护多组逻辑上隔离的键空间, 各自拥有独立的 MemTable、SST 集合和压缩配置, 但共享 WAL。对文件系统而言, 这意味着 inode 基础属性、目录项、增量日志、系统元信息不必在同一种访问模式下被统一处理。

RucksFS 的元数据层依赖这一特性进行分层配置: inodes 列族承担以点查为主的 inode 属性访问; dir_entries 列族承担按父目录前缀扫描的目录项访问; delta_entries 列族承担高频的增量追加和后台折叠; system 列族仅存放 inode 分配器等低频元信息。拆分之后, 不同列族可以分别配置 Bloom 过滤器、前缀提取器和压缩策略, 而无需在一个全局配置上折中。

这种做法也有代价。列族数量增加后, 内存中的 MemTable 和后台 Compaction 状态也相应增加; 跨列族的一致性更新需要依靠事务或批量原子提交来保证。RucksFS 最终只保留了 4 个列族, 是在“访问模式隔离”和“系统复杂度”之间的折中。

2.2.3 WriteBatch、事务与并发控制

RocksDB 在一致性相关的能力上分两层。底层是 WriteBatch, 能够保证一组 put/delete 原子提交; 上层是 TransactionDB, 除原子提交外还提供 get_for_update、锁等待超时、冲突检测等事务能力^[37]。对普通 KV 业务, WriteBatch 通常已足够; 但文件系统元数据有所不同——许多错误恰恰发生在“检查条件”和“执行修改”之间。

以 create(parent, name) 为例。WriteBatch 可以将“写入新 inode”和“插入新目录项”做成原子提交, 但若提交前没有锁定同名目录项键, 就无法阻止另一个事务在同一时间创建同名文件。RucksFS 因此选用 TransactionDB: 先用 get_for_update 锁定相关的 inode 键和目录项键, 再在同一事务内完成条件检查和批量更新, 最后通过事务提交保证跨列族修改的一致性。

从并发控制角度看, 这正是悲观并发控制 (Pessimistic Concurrency Control, PCC) 的思路。目录是否为空、目标名字是否存在、某个 inode 是否还能删除, 这些判断一旦离开事务边界, 就会留下 TOCTOU (Time-of-Check to Time-of-Use)

窗口。RocksDB TransactionDB 的 `get_for_update` 允许调用方在读取键时顺便获取排他锁，把“检查条件”和“执行修改”收拢到同一事务上下文内。

锁粒度同样重要——RucksFS 使用 RocksDB 的行级锁，将冲突范围限定在具体的 inode 键和目录项键上，而不是锁定整棵目录树。`create(parent, name)` 主要锁定目标目录项键和必要的父目录状态，`rename` 则锁定源目录项、目标目录项和相关 inode。只有真正共享这些键的并发操作才会相互阻塞。为了避免死锁，还需规定统一的加锁顺序；RucksFS 在后续实现中采用“按 inode 编号排序后依次加锁”。

这一点也是 RocksDB 相比早期 LevelDB 方案更适合文件系统元数据的原因。TableFS^[2] 验证了 KV 元数据的可行性，但没有原生事务接口，许多并发正确性问题只能推到上层锁去处理。RucksFS 对目录操作的设计正是建立在“RocksDB 已提供可靠的悲观事务能力”这一事实之上。

2.2.4 Bloom 过滤器与前缀迭代器

Bloom 过滤器用于快速判断某个键是否“不存在于某个 SST 文件中”，从而避免无谓的磁盘访问；前缀迭代器则要求一组具有共同前缀的键在字节序上连续排列，以便对同一前缀做局部扫描。对文件系统目录项而言，这两个特性组合起来尤其关键。

若把目录项键设计为“父 inode + 文件名”的拼接形式，同一父目录下的全部子项就天然共享固定前缀。这样一来，`lookup(parent, name)` 可以退化为点查，`readdir(parent)` 则可以退化为前缀扫描。RucksFS 之所以采用大端序编码，也是为了让 inode 的数值顺序与字节序保持一致，从而使前缀范围查询在 RocksDB 中可以直接依赖字节比较器实现。第 3.3 节讨论键编码时，将在这一基础上给出更具体的映射方案。

2.3 POSIX 文件系统语义

一个文件系统只要对外暴露 POSIX 接口，就需要遵守一整套关于 inode、目录结构、权限位和错误码的约定，不能只关注“数据能否读写”。本文并不追求实现 POSIX 的每个细节，而是聚焦于普通文件、目录和常用元数据操作这几部分，使其与 Linux 用户空间程序的预期尽量一致。

2.3.1 inode 模型与目录结构

POSIX 文件系统以 `inode` 作为对象的持久标识。目录并不是一组路径字符串的集合，而是“名字到 `inode`”的映射表。路径解析实质上是从根目录开始逐级查询目录项，再根据目录项获取下一层 `inode`。普通文件、目录、符号链接共用同一种基本身份表示，只是 `mode`、`nlink` 和数据解释方式不同。

这对 RucksFS 有两点直接影响。其一，路径树不能直接存为“全路径字符串到内容”的映射，否则 `rename` 就会退化为对整棵子树的递归重写；其二，目录项和 `inode` 属性需要在存储上分开，否则 `lookup`、`readdir`、`getattr` 都难以高效实现。第 3.3 节的“以边为中心”键编码，正是把目录树拆解为“父目录 `inode` + 子名字”这类基本边关系。

2.3.2 核心文件操作语义

`create`、`mkdir`、`unlink`、`rmdir` 和 `rename` 是本文最关心的 POSIX 操作。它们共同的难点不在于“写几条记录”，而在于必须同时满足若干一致性条件。表 2-3 对这些操作的核心约束做了归纳。

首先，目录项相关操作必须保持名字唯一。若同一父目录下已存在名为 `foo` 的目录项，再次 `create(parent, "foo")` 必须返回 `EEXIST`，而不能出现两个同名条目。其次，`rmdir` 只能删除空目录；若目标目录仍包含子项，必须返回 `ENOTEMPTY`。再次，`rename` 要求原子可见：外部观察者不应看到“旧名字已删除、但新名字尚未建立”的中间状态。

时间戳和链接计数同样属于语义的一部分。普通文件的 `create` 和 `unlink` 会更新父目录的 `mtime/ctime`；`mkdir` 和 `rmdir` 除更新时间戳外，还会改变父目录的 `nlink`。如果这些字段在高并发下仍采用读-改-写方式，热点目录上很快就会出现严重争用。这也是本文引入增量更新机制的直接动机之一。

最后，打开文件与删除目录项之间的关系不能被忽略。POSIX 允许一个文件在目录项被移除后继续被已打开的句柄访问，直到最后一个句柄关闭为止。这要求元数据层区分“名字是否还在目录里”和“对象是否还能被已有句柄引用”两个状态。RucksFS 在这一点上采用延迟删除策略：当 `nlink` 降到 0 但文件仍被打开时，只删除目录项而不立即清除数据，待 `release` 发生后再完成最终删除。

表 2-3 核心 POSIX 目录操作的语义约束

操作	典型返回	关键语义约束
create	成功或 EEXIST	同一父目录下名字必须唯一；普通文件创建会更新父目录的 mtime/ctime
mkdir	成功或 EEXIST	新建对象类型为目录；除更新时间戳外，还会使父目录 nlink 增加 1
unlink	成功或 ENOENT/EISDIR	删除的是普通文件目录项；会更新父目录时间戳；若文件仍被打开，则目录项删除与对象回收需要分离
rmdir	成功或 ENOTEMPTY	只能删除空目录；成功后父目录 nlink 减少 1，并更新 mtime/ctime
rename	成功或相关错误码	源目录项删除与目标目录项建立必须原子可见；跨父目录移动目录时需要同步调整新旧父目录 nlink

2.3.3 权限模型

POSIX 权限模型由 uid、gid 和 mode 三部分组成。FUSE 文件系统既可以在用户态自行实现访问控制，也可以把常规权限判断交给 Linux VFS。RucksFS 采用后一种方式：挂载时启用 default_permissions，由内核在请求进入用户态之前完成基础权限检查。这样做的好处是，常规的 owner/group/other 语义不必在 FUSE 守护进程中重复实现；用户态可以更专注于元数据事务和存储路径本身。

这并不意味着元数据层可以忽略权限字段。文件创建和目录创建仍然必须正确记录 uid、gid 和 mode，否则内核后续的权限判断就失去依据。因此客户端在处理 create 和 mkdir 时会请求发起者的身份信息一并传递给元数据服务。

2.4 本章小结

本章介绍了 RucksFS 依赖的三项基础。FUSE 提供了将 Linux VFS 请求交到用户态客户端的标准通道；RocksDB 以 LSM-tree、列族与事务这三件事共同撑起了适合元数据负载的持久化引擎；POSIX 则规定了目录操作、链接计数、时

间戳和权限模型的行为规范。下一章将在此基础上进入 RucksFS 自身的系统设计与实现。

3 元数据组织与事务模型

RucksFS 是一个用 Rust 实现的分布式文件系统原型。系统由三部分组成：FUSE 客户端提供 POSIX 入口，MetadataServer 负责命名空间和 inode 元数据，DataServer 负责文件数据的读写；三者之间通过 gRPC 通信。其中 MetadataServer 承担了元数据路径上的关键设计，本章先把这部分内部结构讲清楚；客户端如何在 MetadataServer 和 DataServer 之间协调、DataServer 的数据路径如何组织、FUSE 层的可见语义与错误处理，则统一放到第 4 章展开。

本章围绕三件事：目录项如何映射到 RocksDB 键空间，目录修改在并发下如何保持正确性，以及核心 POSIX 操作如何在这套键编码和事务机制上实现。

3.1 设计目标

在进入具体实现之前，需要先把几项从一开始就希望系统具备的性质确定下来，它们会在后续章节反复影响具体决策。

接口层面保留标准的 POSIX 路径。应用程序通过普通的 `open()`、`rename()`、`unlink()` 等系统调用访问挂载点，不需要为 RucksFS 改写任何代码。选择 FUSE 作为对外接口层的直接原因即在于此。

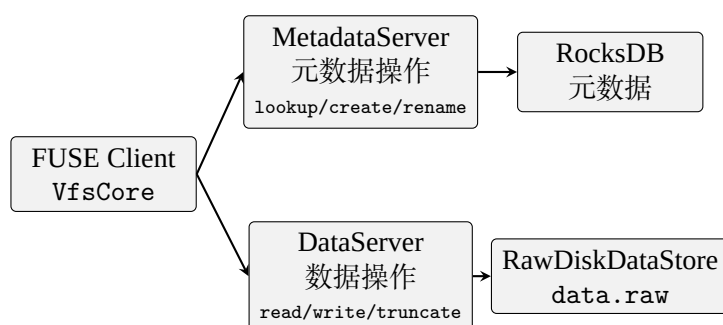
服务层面将元数据与文件数据显式分离。二者的访问模式并不相同：元数据多为小粒度随机写入，且与事务、并发控制紧密相关；文件数据以顺序 I/O 为主，读写路径更短。在服务边界上将两者拆开，既让各自的实现可以按自身特点组织，也使本文真正想研究的元数据路径不被数据块管理的细节干扰。

并发正确性方面需要经得起检验。名字唯一、目录非空判断、跨目录重命名的原子性、延迟删除等语义不能依赖“多数情况下不会冲突”这种经验假设，需要用事务边界把它们固定下来。这一点是采用 RocksDB 悲观事务的直接动机。

性能瓶颈方面，本文希望把可能成为瓶颈的点集中到少数关键路径上——目录项查找、事务提交与父目录属性更新。这几条路径之所以值得单独优化，是因为其行为直接决定了高并发下的吞吐；其他路径即便暂未细致调优，也不会影响整体结论。

3.2 系统整体架构

RucksFS 的运行由三个角色组成:部署在客户端机器上的 FUSE 客户端、负责命名空间和 inode 状态的 MetadataServer,以及负责文件内容读写的 DataServer。客户端收到内核的 FUSE 请求后,并非统一转发到同一个后端,而是先经过本地的 VfsCore 路由层,再按请求类型决定交给元数据面还是数据面。图 3-1 给出了整体的请求路径。



从请求走向来看,这种拆分对应三类路径。

第一类是元数据路径。lookup、create、mkdir、unlink、rename、readdir 等请求经 FUSE 客户端进入 VfsCore,随后被转交给 MetadataServer,再由其访问 RocksDB。

第二类是数据路径。read、write、truncate、flush 等请求由 VfsCore 选定对应的 DataServer,交由原始磁盘后端执行。

第三类是协调路径。有些操作天然跨越元数据面和数据面。例如客户端写完数据后需要通知 MetadataServer 更新文件大小和 mtime;又例如 unlink 或覆盖式 rename 完成后,MetadataServer 会在响应结构中告知客户端哪些 inode 需要在 DataServer 侧清理数据。

客户端不直接持有 RocksDB,也不直接管理原始磁盘布局,只负责在 FUSE 请求、MetadataServer、DataServer 三者之间完成调用串联和结果汇总。这样做的直接好处是,文件系统的正确性主要集中在元数据事务设计和客户端协调逻辑内,模块边界清楚,出现问题时也更容易定位到具体的服务。

整体布局确定之后,本章后续几节按“元数据在 RocksDB 键空间中如何组织”、“事务模型如何保证并发正确性”、“核心 POSIX 操作如何映射到键空间和事

务上”的顺序展开；“客户端如何协调元数据面和数据面”这一部分留到第 4 章。

3.3 元数据模型与存储布局

本节先给出面向实现的元数据整体布局，再分别展开列族划分、键编码和值编码方面的细节。整体布局部分只回答三个问题：系统保存哪些对象，这些对象分别落在哪个列族，以及它们的 key/value 编码是什么样子。至于为何这样划分、这种编码对各类操作意味着什么，留到后面的小节展开。

3.3.1 元数据方案总览

RucksFS 以一棵目录树作为命名空间的抽象模型。用户看到的是按目录组织的层次结构，但真正需要持久化的核心对象只有两类：inode 和 dentry（目录项）。inode 代表文件系统对象本体，保存类型、权限、大小、链接计数、时间戳等属性；dentry 代表目录树上一条父子边，记录“某个父目录下的某个名字指向哪个 inode”。普通文件、目录、符号链接都先由 inode 标识，再通过 dentry 挂接到目录树中。

由此，路径本身不是单独保存的对象，而是通过一系列 dentry 逐级串联得到。以 /docs/paper.tex 为例，先在根目录下找 docs 对应的 dentry，取得目录 inode；再在该目录下找 paper.tex 对应的 dentry，最终定位到目标 inode。围绕这两个核心对象，RucksFS 在 RocksDB 中分别组织 inode 记录和 dentry 记录，并补充数据位置、属性增量和系统状态等辅助元数据。图 3-2 给出这一方案的整体视图。

图 3-2 按列族组织：inodes 集中保存 inode 相关记录，dir_entries 单独保存目录项，delta_entries 保存属性增量，system 保存低频系统状态。后续各小节在此基础上展开各类记录的编码细节与设计考量。

3.3.2 列族划分与设计考量

有了上述的整体布局，下一个问题是：为何这些对象要如此分组，而不是全部放入一个统一键空间？本文的做法是按访问模式将 RocksDB 实例拆分为 4 个列族——inodes、dir_entries、delta_entries、system——分别承担 inode 点查、目录项存储、属性增量追加、系统状态保存这几类不同任务。

inodes 列族承担“对象本体”的点查负载。inode 基础属性、DataLocation

inodes 列族

inode 属性记录: [I] [inode: u64 BE] → 57 字节定长 InodeValue

数据位置记录: [L] [inode: u64 BE] → DataLocation

符号链接记录: [S] [inode: u64 BE] → 目标路径字符串

dir_entries 列族

目录项记录: [D] [parent: u64 BE] [name: UTF-8]

→ [child_inode: u64 BE] [child_mode: u32 BE]

delta_entries 列族

属性增量记录: [X] [inode: u64 BE] [seq: u64 BE] → DeltaOp

system 列族

系统状态记录: 系统保留键 → 分配器状态等元信息

图 3-2 RucksFS 元数据记录与列族布局总览

和符号链接目标在语义上并不完全同类，但访问模式一致——都是按 inode 直接定位的点查——因此放在同一列族中。这样就无需为少量辅助元数据额外维护一套 MemTable 和压缩状态。

dir_entries 列族承担命名空间的边关系。lookup 需要按“父 inode + 名字”定位单条目录项，readdir 需要按父目录枚举全部子项，这两种访问都依赖前缀局部性。因此该列族启用了 9 字节的前缀提取器，让同一父目录下的目录项按前缀连续排列。相比“目录项和 inode 混在同一列族”，这种方式更有利于目录遍历和前缀 Bloom 过滤器生效；相比“使用完整路径作为键”，又避免了跨目录 rename 对整棵子树的改写。

delta_entries 列族承担热点目录的增量写入。其目标不是长期保存完整状态，而是承接高频的 mtime/ctime/nlink 变化，避免每次目录修改都重写父目录 inode 基值。由于该列族中的条目短小、生命周期也短，并且会被后台折叠线程定期消费，这里刻意关闭了压缩，以省去热路径上的编解码开销。

system 列族仅存放 inode 分配器等低频系统元信息。单独隔离是为了避免它与热点目录项共用一套压缩和缓存行为。访问频率虽然远低于其他列族，但它仍需与普通元数据一起走原子提交，因此留在同一个 RocksDB 实例内，不额外引入外部存储。

综上，这套列族划分按访问模式组织布局，而不是按语义模块简单切分。列族本身也是元数据关键路径优化的一部分，因此先把“为什么这么分”讲清楚，再讨论“每类记录如何编码”。

3.3.3 键编码与目录映射

在列族边界确定之后，接下来要解决的问题是：目录树中的 inode 和目录项，如何映射为 RocksDB 中可比较、可扫描、可点查的字节串键。RucksFS 对所有多字节整数统一采用大端序编码，以保证数值顺序与字节序顺序一致；不同类型的键再通过单字节前缀区分，其中 [I] 表示 inode 属性记录，[D] 表示目录项记录，[X] 表示增量记录，[L] 表示数据位置记录，[S] 表示符号链接目标记录。在此基础上，表 3-1 给出几类核心键的具体编码格式。

表 3-1 核心键编码格式

键类型	编码格式	说明
inode 键	[I] [inode: u64 BE]	存放 inode 基础属性
目录项键	[D] [parent: u64 BE] [name: UTF-8]	存放父目录到子名字的映射
增量键	[X] [inode: u64 BE] [seq: u64 BE]	存放时间戳或 nlink 增量
数据位置键	[L] [inode: u64 BE]	存放 DataLocation
符号链接键	[S] [inode: u64 BE]	存放符号链接目标路径

图 3-3 以路径 /docs/paper.tex 为例，展示了目录树如何映射到 RocksDB 键空间。核心思想不是存“整条路径”，而是存“父目录到子名字”的边关系。

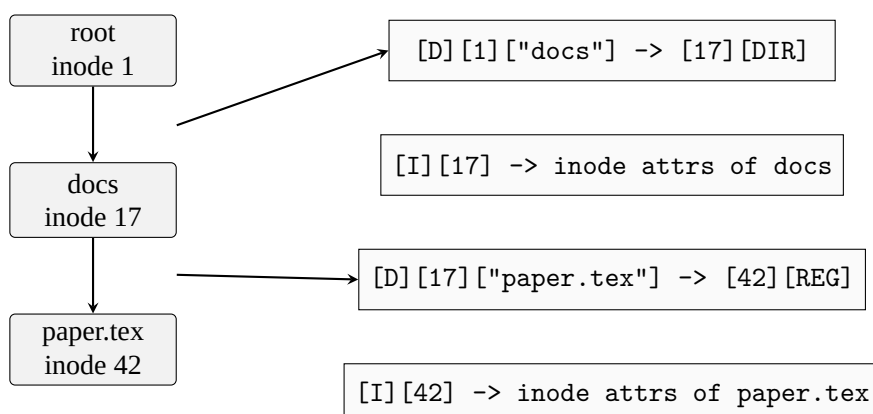


图 3-3 路径树到 RocksDB 键空间的映射示意

这种设计遵循的是“以边为中心”的命名空间建模思路。目录树中的每一条父子边，都对应 dir_entries 列族中的一条记录。lookup(parent, name) 退化

为点查一条目录项键；`readdir(parent)` 退化为扫描同一前缀下的全部目录项键；`rename` 则退化为删除旧边、插入新边。与使用全路径作为键相比，这种建模方式避免了跨目录重命名时对子树的全量重写。第 5.4 节中 `pjdfstest` 的 `rename`、`readdir`、`lookup` 用例将直接检验这一建模方式的正确性。

与 TableFS 键编码方案的对比。 最早将 KV 存储应用于文件系统元数据的 TableFS^[2] 同样采用“父 inode + 文件名”作为目录项键。但 TableFS 使用原生字节序编码 inode 编号，导致同一父目录下的所有目录项在 LSM-tree 的键空间中不一定物理相邻——因为 LevelDB 按字节序比较键，inode 编号 256 的字节序表示为 `\x01\x00\x00\x01`，inode 编号 257 表示为 `\x01\x00\x00\x02`，两者之间可能隔着大量其他 inode 的键。RucksFS 采用大端序编码，使 inode 编号的大小顺序与字典序一致，同一父目录下的所有子项天然在键空间中连续排列。这一差异在 `readdir` 场景下尤为关键：TableFS 需要在扫描时额外过滤父目录编号，而 RucksFS 只需一次前缀迭代即可获得全部目录项。此外，大端序也使得前缀 Bloom 过滤器在同一父目录内的假阳性率更低。

3.3.4 Value 编码与定长序列化

在实现层，inode 的基础属性由 `FileAttr` 抽象表示，覆盖 inode 编号、文件大小、模式位、链接计数以及访问、修改和状态变更时间戳这几项上层接口最常使用的字段。`MetadataServer` 在 RocksDB 中并不使用通用序列化框架，而是按手工布局对这些字段进行定长编码：`InodeValue` 共 57 字节，首字节保留为格式版本号，之后依次是 inode 编号、文件大小、模式位、链接计数和三类时间戳。定长布局有两点直接好处：读写路径只需按偏移切片，无需承担变长解码开销；调试时也可以直接从 RocksDB 中观察二进制布局，问题定位更直接。代价是模式演进需要显式处理——若未来确实需要扩展字段，可以借首字节的版本号引入兼容逻辑，而不必打破既有键空间。

目录项与增量记录的 `value` 另作设计。目录项 `value` 固定为 12 字节的 `[child_inode: u64 BE][child_mode: u32 BE]`，把 `mode` 和子 inode 编号放在一起，使 `readdir` 在拿到目录项时即可判断子对象类型，无需再回查一次 inode。增量 `value` 采用短小的 `DeltaOp` 编码，以使热点目录上的属性更新尽可能表现为

小对象追加写。

3.4 事务模型与元数据一致性

3.4.1 事务边界与原子提交

RucksFS 的大多数元数据操作都不是单键更新。以 `create` 为例，它至少需要同时修改目录项、目标 `inode`、数据位置以及父目录属性；`rename` 往往同时涉及源目录、目标目录、被移动对象和潜在被覆盖对象。因此事务模型首先要回答的不是“如何加锁”，而是“哪些修改必须作为一个不可分割的整体提交”。

本文的做法是不将 RocksDB 原生事务接口直接散落在业务代码中调用，而是通过 `StorageBundle` 和 `AtomicWriteBatch` 将跨列族写入封装起来。调用方先使用 `StorageBundle::begin_write()` 开启一个事务上下文，向其中追加 `BatchOp`，最后统一提交。这样做的好处是，上层逻辑可以围绕“目录项是否存在”、“应该写哪些 `inode` 记录”、“父目录需要追加哪些增量”来组织动作，无需显式处理不同列族的底层细节。提交时底层最终会调用 RocksDB 事务的 `commit()`，把跨 `inodes`、`dir_entries`、`delta_entries` 的修改作为一个原子单元持久化。

3.4.2 悲观并发控制与冲突处理

仅有原子提交还不够，并发目录操作的正确性同样需要处理。这里使用的是 RocksDB TransactionDB 的悲观并发控制（PCC）接口，关键操作是 `get_for_update`：事务在读取目录项键或 `inode` 键的同时获取排他锁，使“检查条件”和“执行修改”落在同一个事务视图内。对文件系统而言这一点较为重要——`create`、`rmdir`、`rename` 都存在典型的 TOCTOU 风险，如果先在事务外检查、再进入事务修改，就会留下竞态窗口。

多对象操作还会带来死锁。本文的做法是：凡可能涉及多个 `inode` 的事务，都先将相关 `inode` 编号收集并排序，再按升序逐个加锁。`rename` 是最典型的例子——源目录、目标目录、被移动对象、潜在被覆盖对象都可能同时进入同一个事务。统一的加锁顺序将“先锁谁、后锁谁”确定为一条规则，不同线程就不会因加锁顺序不一致而形成循环等待。当前实现还为锁等待设置了 5 秒超时；若事务冲突一时无法消解，`MetadataServer` 会先在服务端有限重试，最多 3 次，并采用指数退避；重试仍失败时，才以 `TransactionConflict` 向上返回，在 FUSE

层映射为 EAGAIN。

3.4.3 增量更新与读写一致性

事务并不能解决所有热点问题。目录修改中还有一个较为隐蔽的瓶颈：父目录属性更新。按 POSIX 语义，`create`、`unlink`、`mkdir`、`rmdir` 都会动到父目录的 `mtime/ctime`，涉及子目录的操作还会动 `nlink`。如果每次都直接读出父目录 `inode` 再写回，热点目录下的冲突就会集中到同一条基值记录上，LSM-tree 的顺序追加优势也无法体现。

本文的做法是把这类更新从“基值重写”改为“增量追加”。写路径不直接修改父目录在 `inodes` 中的基础值，而是把变化编码为一条 `DeltaOp`，追加到 `delta_entries` 中。目前的增量类型有 `IncrementNlink`、`SetMtime`、`SetCtime`、`SetAtime`。普通文件的创建和删除只影响父目录时间戳；`mkdir`、`rmdir` 和跨目录移动目录时会额外引入 `nlink` 增量。这样热点目录上的高频更新就从“反复覆盖同一条 `inode` 基值”变为“向独立的增量记录追加”。

读取路径需要负责把“基础值 + 增量”重新合成为最新视图。`MetadataServer` 在加载 `inode` 时，优先从 `InodeFoldedCache` 读取已经折叠好的结果；缓存未命中则先读取 `inodes` 中的基础值，再扫描属于该 `inode` 的 `DeltaOp`，按顺序合并。为了避免读路径的折叠成本无限增长，后台的 `DeltaCompactionWorker` 会周期性检查 `dirty inode`，当增量条目超过阈值时把折叠后的新基值写回 `inodes`，同时删除已消费的增量。当前默认阈值 32 条、检查周期 5 秒。其思路可以概括为：用可控的读路径折叠成本，换取写路径上的冲突减少。和 `Mantle` 的 `Delta Record` 相比，本文的 `DeltaOp` 沿用了“以增量换写放大”的基本思路，但将追加、折叠和提交都限制在同一个 `RocksDB` 实例内完成，实现上更为轻量。

这套机制的收益在低并发下并不明显：父目录上的写争抢需要达到一定密度，覆盖式更新和增量追加的差距才会拉开。第 5.6 节将使用 `Delta` 和 `NoDelta` 两种编译变体做对照实验以衡量这一点——在共享父目录的元数据负载下，当到达服务端的请求速率跨过事务冲突阈值之后，关闭 `DeltaOp` 的构建吞吐出现明显下降，而开启 `DeltaOp` 的构建仍能继续扩展。

3.4.4 删除语义与句柄生命周期

除了事务原子性，元数据一致性还体现在删除语义上。POSIX 允许目录项已经从命名空间中消失，已打开的句柄仍然能继续访问原文件。仅靠一次 `unlink` 事务不足以完成这件事，因为“名字空间可见性”和“数据回收时机”并不总在同一时刻。本文因此在 `MetadataServer` 内维护了 `open_handles` 和 `pending_deletes` 两类状态。`open()` 成功后对应 `inode` 的打开计数加一；`release()` 时减一。

当 `unlink` 或覆盖式 `rename` 把某个普通文件的 `nlink` 降为 0 时，`MetadataServer` 并不一律立刻删除数据。若该 `inode` 还有打开句柄，系统只将其加入 `pending_deletes`；等最后一个句柄关闭时，再通知客户端到 `DataService` 侧真正清理数据。这样目录项删除、链接计数归零、物理数据回收就分为三个阶段：名字先从命名空间中消失，可达性由 `nlink` 表示，物理删除则推迟到没有进程持有该对象时再执行。从外部来看，这对应了 POSIX 要求的“名字已删除、但已有句柄仍然有效”的语义。`pjdfstest` 中的 `unlink::open_file_not_freed` 用例直接检验了这一行为，结果见第 5.4.4 节。

3.5 核心操作实现

`RucksFS` 的大多数元数据操作走同一条主线：先定位少量相关的目录项键和 `inode` 记录，在同一事务内完成检查和修改，按需向父目录追加属性增量。本节按操作类别逐一说明这条主线如何落到具体流程上，同时也说明前述键编码、`PCC`、`DeltaOp` 三件事如何在这些操作中组合复用。

3.5.1 `create` 与 `mkdir`

`create` 的难点不在于“写入一个新 `inode`”，而在于需要同时满足三个条件：目标名字在此之前不存在；新对象以完整状态出现；父目录的属性更新与前两者保持原子一致。若将这些动作拆成多个独立步骤，就容易出现中间状态——例如 `inode` 已经建立但目录项尚未插入，或者同名检查通过后被其他并发事务抢先插入。算法 3-1 给出当前实现的 `create` 流程。

从键空间组织的角度看，`create` 能够直接体现“以边为中心”建模的好处：不需要改写整条路径，也不需要搬移子树，只需在 `dir_entries` 中插入一条新

算法 3-1 create(parent, name, mode, uid, gid) 的核心流程

- 1: 开启事务 batch
 - 2: 对目录项键 D[parent, name] 执行 get_for_update
 - 3: **if** 目录项已存在 **then**
 - 4: 返回 EEXIST
 - 5: **end if**
 - 6: 分配新 inode 编号, 构造普通文件的 InodeValue
 - 7: 向 inodes 列族写入新 inode 基值
 - 8: 向 dir_entries 列族写入父目录到新文件的映射
 - 9: 向 inodes 列族写入该 inode 的 DataLocation
 - 10: 向 delta_entries 追加父目录的 SetMtime 与 SetCtime
 - 11: 提交事务
 - 12: 更新本地缓存并返回新文件属性
-

边, 并在 inodes 中补齐目标对象的基础记录和数据位置记录。新对象的可见性由一条目录项记录和一组 inode 记录共同决定, 而不是由单一结构承载。

另一方面, 父目录属性并不走“覆盖写回基础 inode”这条路径, 而是追加两条 DeltaOp——一条 SetMtime、一条 SetCtime。这样一来, create 就从“目录项插入 + 父目录基值重写”的组合, 转换为“目录项插入 + 若干新记录写入 + 增量追加”的组合。前者在热点目录下容易将竞争集中到父目录 inode 上; 后者更贴合 LSM-tree 擅长的追加写模式。

mkdir 与 create 共用同一条事务骨架, 但比普通文件创建多了一层目录相关的语义约束。新对象的类型位是目录, 初始 nlink 为 2; 父目录除了更新时间戳之外, 还需要追加 IncrementNlink(+1), 因为多了一个子目录。普通文件创建不会改变父目录 nlink, 而目录创建会——这一差异看似细微, 但直接决定了后续 rename、rmdir 以及一致性检查的正确性。

3.5.2 unlink、rmdir 与 release

若将 create 理解为“建立一条新边”, 则 unlink 和 rmdir 分别对应“删除一条普通文件边”和“删除一条目录边”。两者的共同点是都需要在事务内同时处理目录项可见性、目标对象状态和父目录属性变化; 差异则来自 POSIX 对普通文件和目录所加的不同约束。

`unlink` 的核心流程是：锁定目标目录项，确认其不是目录，删除该条目录项记录，将目标 `inode` 的 `nlink` 减一，再根据链接计数和打开句柄的状态决定是否进入延迟删除分支。关键在于目录项删除和对象状态更新处于同一事务内，不会出现“名字已消失但链接计数尚未更新”这种中间状态。父目录的 `mtime/ctime` 同样不直接重写基值，继续走增量日志。

`rmdir` 的难点在于空目录检查。若写成“先扫描一遍目录、确认为空、再开启事务删除”，检查结果在事务开启前就可能失效。本文的做法是把“是否为空”的判断也放入事务，用一致性视图下的前缀扫描完成。若目录非空，直接返回 `ENOTEMPTY`；若为空，则删除目标目录项和目标 `inode`，并向父目录追加 `IncrementNlink(-1)` 及一组时间戳增量。空目录约束由事务本身保证，不依赖上层约定。

`release` 对应延迟删除的最后一步——物理数据回收。`unlink` 和覆盖式 `rename` 可以使名字从命名空间中消失，但只要仍有进程持有打开句柄，物理数据就不能立刻清理。每次客户端关闭文件句柄时，`release` 都会将打开计数减一；若某个 `inode` 之前已进入 `pending_deletes`，而此时计数恰好降为 0，`MetadataServer` 才会把该 `inode` 返回给客户端，通知它到 `DataServer` 侧完成真正的数据删除。这样，“名字可见性”、“元数据可达性”和“物理数据回收”被清晰地分为三个阶段。

3.5.3 rename

`rename` 是前述几项设计决策综合体现最为充分的操作。它既依赖“以边为中心”的键建模，也依赖统一的加锁顺序、覆盖删除语义以及父目录属性增量，可以视作整条元数据路径中最能体现系统设计取舍的操作。它可能同时牵涉源目录、目标目录、被移动对象、潜在被覆盖对象这四类状态，并且要求整个过程对外原子可见。算法 3-2 概括了这一流程。

从实现角度看，`rename` 的主体可以概括为“删除旧边、插入新边，并按需处理被覆盖对象”。这是边中心建模的直接好处：即便是跨目录移动，系统也只需要改动少量与源路径、目标路径直接相关的记录，而不需要像全路径键那样对子树做级联改写。`rename` 的复杂性主要来自语义约束，而非底层数据搬移。

算法 3-2 rename(old_parent, old_name, new_parent, new_name) 的核心流程

- 1: 读取并锁定源目录项
 - 2: 若目标目录项存在, 则读取其 inode 信息
 - 3: 收集全部相关 inode 编号并排序, 按序执行 get_for_update
 - 4: 检查源对象与目标对象的类型兼容性
 - 5: **if** 目标存在且为非空目录 **then**
 - 6: 返回 ENOTEMPTY
 - 7: **end if**
 - 8: **if** 目标存在 **then**
 - 9: 按覆盖删除逻辑处理目标对象
 - 10: **end if**
 - 11: 删除源目录项, 写入目标目录项
 - 12: 更新被移动对象的 ctime
 - 13: **if** 移动的是目录且跨父目录 **then**
 - 14: 对旧父目录追加 IncrementNlink(-1)
 - 15: 对新父目录追加 IncrementNlink(+1)
 - 16: **end if**
 - 17: 对涉及的父目录追加时间戳增量
 - 18: 提交事务并返回被覆盖对象列表
-

排序加锁是 `rename` 避免死锁的关键，但其意义不止于此。其更关键的作用在于，统一加锁顺序让源目录、目标目录和被覆盖对象可以放在同一个一致的事务视图下，使“检查目标是否存在”、“检查目标目录是否为空”、“删除旧目录项”、“写入新目录项”、“处理覆盖删除”这一系列动作共享同一个一致性视图。在外部观察者看来，`rename` 表现为一次原子切换，不会暴露“旧名字已删除、新名字尚未建立”这类中间状态。

还有一个容易忽略的细节：只有在目录跨父目录移动时才需要调整父目录 `nlink`。移动普通文件不会影响任何目录的 `nlink`；同一父目录内对目录重命名也不会影响父目录 `nlink`。这些差异直接决定了增量日志中需要追加哪些 `DeltaOp`，也说明 `rename` 的正确性不仅取决于“目录项是否改动正确”，还取决于相关目录属性是否被同步维护。

3.5.4 `readdir`、`open` 与写入协调

`readdir` 最能体现键空间的局部性。在给定父目录 `inode` 之后，`MetadataServer` 构造前缀 `[D][parent: u64 BE]`，利用 `RocksDB` 的前缀迭代器即可顺序扫描该目录下的所有 `dentry`。由于目录项 `value` 已包含子对象的 `mode`，普通 `readdir` 无需为每个条目再回查一次 `inode` 即可获取文件类型。由此可见，在目录项 `value` 中额外保存一个 `child_mode` 并非冗余，其目的正是降低目录遍历的读放大。

`open` 是元数据面与数据面的衔接点。它本身不负责数据读写，而是将“该 `inode` 的数据应去哪一台 `DataServer` 读写”这一信息交给客户端。因此 `open` 返回的不只是句柄，还包括对应的 `DataLocation`。这样后续的 `read`、`write`、`truncate` 就不必每次都回 `MetadataServer` 重新询问路由，可以直接访问正确的数据后端。

写入完成后的元数据更新同样体现出 `RucksFS` 的职责划分：`DataServer` 只负责把字节写入正确位置，并不主动维护文件长度或 `mtime`；这些语义统一由 `MetadataServer` 掌握。客户端在 `write_data` 成功之后会显式调用 `report_write(inode, new_size, mtime)` 回写元数据。这样做的好处是边界清晰：数据路径保持简单，元数据语义集中维护；代价是客户端需要多做一次协调动作。客户端如何具体承担这一协调职责，以及 `DataServer` 在另一侧如何组织数据路径，留到第 4 章展开。

3.6 本章小结

本章围绕 inode 与 dentry 这两个核心对象，集中讨论 MetadataServer 内部的元数据组织与事务模型。元数据按访问模式被划分到 inodes、dir_entries、delta_entries、system 四个列族；目录树通过“以边为中心”的键编码映射到 RocksDB 键空间，使 create、unlink、rename 等操作对应到对一条边的增删；事务部分借助 RocksDB 悲观锁和统一加锁顺序保证并发正确性；父目录属性则通过 DeltaOp 增量记录减少热点冲突。在这套键编码、事务与 DeltaOp 的组合下，create、mkdir、unlink、rmdir、rename、readdir、open 等操作都能在同一条主线下落地。

本章的讨论仅限于 MetadataServer 内部。一次完整的用户请求还需要 FUSE 客户端、MetadataServer 与 DataServer 三者协同，这部分跨服务的路由、编排、数据路径、FUSE 可见语义以及错误处理，将在第 4 章单独展开。

4 跨服务协调与系统实现

第3章讨论的是 MetadataServer 内部如何组织元数据、如何用事务保证一组目录操作的原子性，以及如何借助 DeltaOp 缓解父目录的写热点。但对一次真实的用户请求而言，仅有 MetadataServer 内部的正确性还不够。请求从内核 VFS 进入 FUSE 客户端之后，往往需要在 MetadataServer 和 DataServer 之间来回协调才能完成一次完整的 POSIX 语义。本章把视角从单个元数据服务抬到整个分布式系统，依次说明客户端如何进行路由与编排、DataServer 如何承担简化后的数据路径、FUSE 层如何控制内核可见语义、gRPC 接口又如何如何在两侧之间划出明确的模块边界，最后说明正常路径之外的错误处理与恢复机制。

4.1 客户端路由与语义编排

客户端内部的协调层是 VfsCore。它对 FUSE 暴露统一的 VfsOps 接口，同时持有 MetadataOps 和 DataOps 两侧的句柄，负责决定一次请求应该落到哪个服务、是否需要跨服务串联，以及结果返回之后是否还需要补充后续动作。相对于单纯的 RPC 转发器，它更接近整条路径上的编排层。

请求大致分为三类。第一类是纯元数据操作——lookup、create、mkdir、unlink、rmdir、rename、readdir、getattr、setattr——直接交给 MetadataServer。第二类是纯数据操作——read、write、truncate、fsync——在文件已经打开的情况下可以按句柄中缓存的 server_id 直接路由到对应的 DataServer。第三类是跨服务的复合操作，例如 open 需要先从 MetadataServer 获取 DataLocation；release、覆盖式 rename 以及可能触发截断的 setattr 则需要在元数据结果返回后由客户端补充一个数据侧的清理或截断请求。

这样安排的意图较为明确：命名空间和 inode 状态的语义约束在 MetadataServer 内，字节读写、截断、清理的工作留在 DataServer 内，跨两侧的复合语义由客户端显式协调。三者的边界因此较为清晰，客户端充当 MetadataServer 元数据事务和 DataServer 字节读写结果的汇合点，把两侧的结果组合成用户看到的 POSIX 行为。代价是客户端比本地文件系统中通常只做请求转发的客户端更复杂一些，这是分离式架构必须承担的一层成本。

4.2 DataServer 与原始磁盘后端

数据侧本文走的是一条较为简化的路径。DataServer 只是 DataStore 的服务化封装层，当前唯一的后端是 RawDiskDataStore。它把所有 inode 的内容映射到同一个 data.raw 文件，按固定公式

$$\text{disk_offset} = \text{inode} \times \text{max_file_size} + \text{file_offset}$$

计算物理偏移，再用 pread/pwrite 读写。DataServer 并不维护复杂的块映射结构，而是将 inode 编号直接当作数据文件中的一个逻辑槽位。

这种固定地址映射的好处较为直接：数据路径不需要空闲空间位图、extent 树或块分配器，寻址是常数时间，服务端状态也较为简单。对本文聚焦的“元数据关键路径优化”而言还有一项附加好处——数据侧逻辑越简单，第 5 章中的性能差异就越容易归因到元数据组织、客户端协调和网络通信上，不会被复杂的块管理逻辑所干扰。

但这里也需要指出当前实现仍有一处短板。每个 inode 都预留 max_file_size 大小的逻辑地址空间，当前默认 128 MiB，在大量小文件场景下空间利用率较低；delete_data 目前仍为空操作，对象删除后物理空间不会立即回收。空间管理本身并非不重要，只是本文在工程上有意把这些复杂度推迟到后续扩展中，用来换取当前原型在数据路径上的可解释性和实现简洁性。

并发方面，写入偏移由调用方指定，不同客户端向同一 inode 的不同偏移写入，不会因 DataServer 内部地址分配而产生额外竞争；pread/pwrite 也保证每次 I/O 以调用为边界完成。若不同写入请求本身覆盖了重叠区域，该情形属于上层应用的并发语义问题，不是 DataServer 需要额外裁决的状态冲突。

4.3 FUSE 可见语义与缓存策略

在用户态文件系统中，一项操作的可见性能很大程度上取决于内核如何缓存它的元数据。因此在进入实验之前，需要先把 FUSE 层的几项可见语义讲清楚，否则第 5 章中的数据容易被误读。本文主要控制三项：目录项缓存、属性缓存，以及目录遍历接口的选择。

目录项缓存（entry cache）决定内核认为“名字到 inode”的映射在多长时间

内有效。属性缓存 (attr cache) 决定 stat、getattr 等请求是否需要到达服务端。本文当前默认将两者的 TTL 均设为 0，即每次 lookup 和 getattr 都穿透 FUSE 送到 MetadataServer；这一选择直接服务于第 5 章的测量目标——将元数据路径的吞吐归因到 MetadataServer 本身，而不是被客户端缓存遮蔽。在真实生产部署下，这两项 TTL 可以按需放宽到秒级，以降低重复查询的 RPC 开销。目录遍历接口显式启用 readdirplus，让目录遍历在返回目录项的同时附带基本属性，从而省去“先 readdir 再对每个条目逐个 getattr”的往返。

除了缓存以外，FUSE 语义中还有两点与分离式架构较为相关。其一，open 在本文实现中不仅产生句柄，还负责把数据路由信息交给客户端。其二，fsync 目前只保证数据侧 flush，元数据依赖 RocksDB 的事务提交和 WAL 持久化，尚未实现严格的双阶段刷写。换言之，本文在 FUSE 这一层没有把内核接口原封不动地映射到远端服务，而是按元数据面和数据面分离后的职责边界，为每个接口给出了一个可落地的语义解释。

4.4 gRPC 接口与模块边界

gRPC 在这条路径上不仅承担通信，也决定了哪些状态由哪个服务直接返回、哪些后续动作需要客户端继续协调，接口设计因此也是系统结构的一部分。本文在接口层面遵循两条原则：元数据操作和数据操作严格分离；一次 RPC 尽量返回下一阶段所需的全部信息，避免将简单操作拆成多轮往返。

据此，MetadataServer 的接口按职责分为四类：命名空间操作 (Lookup/Create/Rename/Readdir 等)、属性操作 (GetAttr/SetAttr)、句柄操作 (Open/Release) 以及写入后的元数据协调 (ReportWrite)。DataServer 的接口则刻意收敛，只暴露 Read/Write/Truncate/DeleteData 等直接操作字节内容的调用。这样在接口层面即可判断“谁负责对象语义、谁负责数据内容”。

消息格式也服务于同一目标。元数据响应通常直接携带完整的 FileAttr、DataLocation、purged_inodes 或 needs_truncate，使客户端不必为完成一个上层操作而再发一轮探测 RPC。连接层面，当前原型采用“每客户端单 gRPC 通道、共享 HTTP/2 连接”的形式，主要是为了控制实现复杂度；这并非最终形态，但对本文所关心的元数据语义验证和原型级性能评估已经够用。

4.5 错误处理与恢复机制

一旦系统从单进程存储引擎扩展到“客户端 + MetadataServer + DataServer”这样的三段式路径，正确性问题就不仅来自事务冲突，还来自 RPC 失败、服务崩溃和跨服务状态不一致。考虑到第 5 章主要呈现正确性和性能结果，不再解释系统行为本身，这里需要先把故障模型和恢复边界说明清楚。

4.5.1 事务冲突、RPC 失败与错误码映射

最常见的一类失败是 MetadataServer 内部的事务冲突。对这种情况，本文没有选择把 RocksDB 的底层错误直接抛给用户，而是先在服务端进行有限重试：最多 3 次，采用指数退避；只有重试仍然失败，才以 `TransactionConflict` 向上返回，再在 FUSE 层映射为 `EAGAIN`。这样偶发冲突不会直接转化为用户可见错误，错误语义也更为清晰：`EAGAIN` 表示的是“事务冲突尚未消解”，而不是任意 I/O 异常。

另一类失败来自普通的 RPC 或 I/O 调用。当前原型对此选择了较为保守的做法：客户端按 gRPC 返回的错误状态向上报相应的文件系统错误，不会在数据路径上维护待确认写队列，也没有实现基于 `inode + offset` 的幂等重发。也就是说，当前实现只做了“事务冲突自动重试”这一项，跨服务失败并不会被自动屏蔽，而是直接向应用暴露。

4.5.2 服务崩溃后的恢复语义

MetadataServer 崩溃时，它持有的 RocksDB TransactionDB 会依赖 WAL 完成恢复：已经提交但尚未刷入 MemTable 的内容会在重启后恢复，而尚未完成提交的事务不会被当作“已经成功”。因此，只要某次元数据操作已经向客户端返回成功，其持久化结果就不会因 MetadataServer 进程崩溃而丢失。客户端侧会把崩溃表现为一次 RPC 失败或重试；必要时，`open` 这类操作可以重新向 MetadataServer 请求新的 `DataLocation`。

DataServer 的崩溃语义更为简单，但也受当前实现边界的约束。若负责某个文件的 DataServer 不可达，对应的 `read` 或 `write` 会直接向上返回 `EIO`。由于当前原型未引入数据副本或多 DataServer 故障切换，数据服务故障不会被透明屏

蔽，而是直接暴露为“该文件暂时不可访问”。又由于 `delete_data` 目前仍为空操作，删除路径也就不存在“删除已提交但物理空间尚未回收”这类额外的恢复问题。

4.5.3 当前实现边界与论文讨论范围

讲完正常路径与故障路径之后，需要明确本文的讨论边界。当前 RucksFS 有意保持“单 MetadataServer + 单默认 DataServer”这条主路径。虽然 `DataLocation` 和 `VfsCore::with_data_servers()` 已经为多 DataServer 路由预留了接口，但元数据分片、多副本一致性、物理空间回收以及严格的双阶段 `fsync` 等都尚未实现。

本文的研究重点是元数据面和数据面分离之后，如何通过键编码、事务控制、客户端协调和增量更新来优化元数据关键路径，而不是构建一套完整的大规模分布式控制面。相应地，第 5 章的实验也主要在当前边界内验证这套设计的正确性和性能特征，不声称系统已经覆盖多副本容错、跨节点故障转移或空间回收等更大范围的工程能力。

4.6 本章小结

本章把视角从第 3 章的单一元数据服务抬到了整个分布式系统。客户端的 `VfsCore` 作为路由与编排层，按请求类别将调用分发给 `MetadataServer`、`DataServer` 或两者的组合；`DataServer` 以 `RawDiskDataStore` 这一最小实现承担字节读写，换取原型阶段数据路径的简洁性和可解释性；`FUSE` 层的目录项/属性缓存与 `readdirplus` 选项，连同 `open` 返回数据路由、`fsync` 仅保证数据侧刷写等可见语义，共同决定了内核看到的文件系统行为；`gRPC` 接口则在两侧之间划出明确的职责边界，使客户端能用尽量少的 `RPC` 往返完成一次完整的 `POSIX` 操作。

错误处理部分沿着“事务冲突—`RPC/IO` 失败—服务崩溃”三层展开：`MetadataServer` 内部冲突由服务端有限重试消化，跨服务失败目前仅向上报错，未实现自动屏蔽；`RocksDB WAL` 为 `MetadataServer` 提供了崩溃一致性保证，`DataServer` 侧的不可达则直接以 `EIO` 暴露。本文明确把多副本、元数据分片、空间回收和严格双阶段 `fsync` 划在讨论范围之外，作为后续工作。第 5 章的实验

将在这一主干的基础上，用 pjdftest 合规性、mdtest 横向对比以及 Delta/NoDelta 两个构建变体的对照，检验第 3 章和本章的各项设计是否达到了预期。

5 测试与结果分析

第 3 章已经说明了 RucksFS 在设计上的元数据组织方案、事务模型和增量更新机制。本章则需要把这些设计放到真实分布式部署下，检验其是否达到预期。

5.1 实验目的与测量范围

本文的研究对象是元数据键值存储层面的文件操作，具体落到系统上就是 RucksFS 的 MetadataServer（后文简称 MDS）——它承担目录项编码、inode 属性维护、事务提交、DeltaOp 增量更新等全部元数据逻辑。本章性能实验的测量对象即 MDS 本身的处理能力，所关心的是一次元数据操作从 FUSE 请求进入到服务端事务提交再返回这条路径上的真实延迟和吞吐。

基于这一测量范围，对客户端缓存和服务端缓存需分别处理。客户端侧，内核 VFS 在收到 FUSE 返回的属性 TTL 之后会在本地维护一份属性缓存，命中时 stat 请求不经过 FUSE 守护进程，也不到达 MDS，对 MDS 处理能力的测量没有贡献；NFS 客户端的 attr cache 以及 JuiceFS 客户端的 attr/entry/dir-entry cache 与此性质相同。因此本章将三套被测系统的客户端属性缓存全部关闭——RucksFS 通过将 FUSE 返回的 entry/attr TTL 设为 0 实现，JuiceFS 通过 `--attr-cache=0 --entry-cache=0 --dir-entry-cache=0` 实现，NFS 通过 `noac` 挂载选项实现。服务端侧的缓存（MDS 的 InodeFoldedCache、RocksDB BlockCache、TiKV 的 BlockCache）属于元数据引擎设计的组成部分，保留默认配置，由测试正常触发其命中行为。

本章的对比分为三组，各自承担不同角色。第一组是 RucksFS-Delta 与 RucksFS-NoDelta 的内部对比——两者仅差 `batch_parent_deltas` 一个编译期开关，其余代码完全一致，用于单独考察 DeltaOp 增量更新机制在共享父目录高并发下的作用。第二组是 RucksFS 与 JuiceFS+TiKV 的横向对比，两者都是 FUSE 客户端加 KV 后端的组合，区别在于 RucksFS 采用本机直连 RocksDB 的专用元数据引擎，JuiceFS+TiKV 采用通用分布式事务 KV 作为元数据后端；这一组的目的是衡量“针对元数据访问模式做过定制的专用 KV 后端”与“通用分布式

KV 后端”两条工程路线的性能差异。第三组是 RucksFS 与 NFS v4.2 的横向对比，NFS 服务端底层是 ext4 的目录 B+ 树；这一组的目的是衡量“KV 作为元数据存储模型”与“传统本地文件系统作为元数据后端 + 网络服务化”两条路线的差异。

这里需要解释一下第三组为何选择 NFS，而不是直接在本机上测 ext4。本文的研究对象是“把目录项和 inode 建模为 KV 记录”这一元数据存储模型，不是文件系统的整体拓扑；要让 KV 方案与传统目录 B+ 树方案的差距能够清楚归因到存储模型本身，两侧的上层拓扑必须尽量对齐。RucksFS 是“FUSE 客户端 + 网络 RPC + 远端服务”的形式，若直接与本机 ext4 对比，差距中会混入“用户态 vs 内核态”“有无网络 RPC”两层额外变量，ext4 的实测数据会被额外的优势——跳过 FUSE 用户/内核切换、不经过网络——整体抬高，难以支撑对“存储模型”本身的结论。NFS v4.2 服务端底层正是 ext4 的目录 htree，但整体形态同样是“客户端 + 网络协议 + 远端服务”，两侧只在元数据存储模型这一项上有差异，其余可以认为大致对齐。本章因此选择 NFS v4.2 作为“非 KV 元数据方案”的参照。

这一测量范围同样限定了本章可以做出和不能做出的断言。本章能够支撑的断言是：在关闭客户端 VFS 缓存、MDS 自身负责处理每一次元数据请求的前提下，RucksFS 的 MDS 路径相对上述两条常见工程路线具有怎样的吞吐与扩展性表现。本章不做也无法做的断言是：包含客户端 VFS 缓存在内的应用视角完整性能比较——那需要将三套系统在生产默认配置下再测一轮，属于后续工作。

5.2 实验组织

实验分为两条线。一条线是 POSIX 语义合规性。使用 pjdftest 套件在分布式部署环境下（客户端和服务端分别位于不同节点）运行 398 个用例，覆盖 13 类系统调用，检验 RucksFS 在 FUSE + RocksDB 这套组合之上是否能够正确表达 POSIX 接口的行为。

另一条线是分布式场景下的元数据性能，按上一节所述的三组对比组织。一方面将 RucksFS 与 NFS v4.2、JuiceFS+TiKV 放到同一套硬件上一同压测，观察横向位置；另一方面利用编译期特性开关得到 Delta 和 NoDelta 两个构建变体，

通过自对比单独考察 DeltaOp 机制的效果。

两类实验共用同一套硬件和工具，数据之间可以直接相互引用。在展开具体实验之前，表 5-1 先将第 3 章的关键设计点与本章用于验证这些设计的实验列出来，后续各节可以与该表对照阅读。

表 5-1 设计要点与实验验证的对应关系

第 3 章设计要点	验证实验	对应章节
边中心键建模 + 大端序编码	pjdfstest 中 rename、readdir、lookup 用例	5.4
PCC 事务 + 统一加锁顺序	pjdfstest 中并发语义用例	5.4
删除语义与句柄生命周期	unlink::open_file_not_freed	5.4
DeltaOp 增量追加	Delta vs NoDelta 对照	5.6
VfsCore 路由与 FUSE 缓存	全链路 mdtest 性能	5.5

5.3 实验环境

5.3.1 集群拓扑

测试集群部署在腾讯云香港二区 (ap-hongkong-2)，所有节点在同一 VPC、同一可用区里，内网 RTT 不超过 0.5 ms。集群由一台大规格服务器加上若干等规格客户端组成，硬件配置见表 5-2。

表 5-2 测试集群硬件配置

角色	机型	关键参数
Server	SA5.16XLARGE256	64 vCPU、256 GiB DDR5、200 GB CLOUD_BSSD
Client	SA5.MEDIUM2	2 vCPU、2 GiB DDR5、50 GB CLOUD_BSSD

客户端数量按测试档位递增，依次为 $N = 2, 8, 32, 64, 96$

操作系统：Ubuntu 22.04 LTS (kernel 5.15)

服务器选用 64 核大规格机型，目的是让各并发档位下被测系统的元数据后端都不会在测试本身之前先达到 CPU 上限。客户端选用 2 vCPU、2 GiB 内存的最小规格；在关闭 FUSE 属性缓存、每次 create 都要穿透到服务端的条件下，单 rank 在这一规格客户端上的 create 吞吐不超过约 400 ops/s，2 核即足以用满该上限。提高单台客户端的规格并不会改变结果，只会增加成本，同时也会占用

更多配额，不方便扩展客户端数量。所有客户端硬件规格保持一致，以排除横向不可比因素。在 $N = 64$ 与 $N = 96$ 两档下，为了让到达 MDS 的请求速率继续上升以检验 DeltaOp 的作用，本章会在每台客户端上启动多个 mdtest rank，让客户端侧的 FUSE 管道被用满，相关设计在第 5.3.3 节具体说明。

5.3.2 被测系统

本章设置了三套被测系统，加上 RucksFS 自身的两种构建变体；每次只运行一套，切换时完全清场。

- **RucksFS-Delta**: 默认构建，启用第 3 章第 3.4 节描述的 DeltaOp 增量追加机制。父目录的 mtime、ctime、nlink 更新走增量路径。
- **RucksFS-NoDelta**: 编译期通过 cargo 特性开关关闭 DeltaOp。父目录属性更新退化为对父 inode 基值的读-改-写，其余代码与 Delta 变体完全相同。该变体仅用于对照，并非生产构建。
- **NFS**: Linux 内核 NFS v4.2 服务端 (nfs-kernel-server)，导出本地 ext4 目录。客户端以 vers=4.2,noac 挂载。noac 的作用是关闭客户端属性和目录项缓存，使每次 stat 都到达服务端，与 RucksFS 将 FUSE TTL 设为 0、JuiceFS 将三类缓存全部设为 0 的做法保持一致。此外，默认配置下 NFS 客户端缓存的存在会导致 mdtest 在多客户端共享父目录场景下出现 dentry 不一致，本节后文第 5.5 节会给出相关观察。nfsd 线程数为 128。
- **JuiceFS+TiKV**: JuiceFS 1.3.1，元数据后端是单节点 TiKV 8.5.6 (含 PD)，数据存储指向同机本地目录。这一组合对应业界常见的“通用分布式 KV + 分层文件系统”路线。JuiceFS 与 RucksFS 共用 RocksDB 作为底层存储引擎 (TiKV 内部同样使用 RocksDB)，便于隔离上层架构差异。

服务器在切换被测系统时执行严格的清场流程：停止所有守护进程，删除数据目录，确认监听端口释放后再启动下一套。客户端侧同样卸载挂载点、清理 FUSE 残留连接并重建挂载目录。整个流程由一个统一的 orchestrator 脚本驱动，确保每次测试都从干净状态开始。

5.3.3 评估工具与执行流程

POSIX 合规性这一线使用 `pjdfstest` (Rust 实现版)，共 398 个用例，覆盖 `chmod`、`chown`、`link`、`mkdir`、`mkfifo`、`mknod`、`open`、`rename`、`rmdir`、`symlink`、`truncate`、`unlink`、`ftruncate` 等系统调用。

性能测试这一线使用 `mdtest 4.1.0`^[38] (IOR 套件的子项，IO500 列表也以此作为元数据基准)，以 OpenMPI 4.1.2 跨客户端调度 MPI 进程。每台客户端挂载一次目标文件系统，并通过 OpenMPI 的 `hostfile` 机制在挂载点上启动一个或多个 `mdtest rank`。本章将并发度记为 $np = N \times r$ ，其中 N 是客户端台数， r 是每台客户端上 `mdtest rank` 的数量， np 也就是同时对服务端发起请求的 `rank` 总数。

具体档位的设计需要结合一个前提：客户端侧三类缓存全部关闭后，单 `rank` 在 2 核客户端上的 `create` 吞吐上限约为 400 ops/s。这一上限来自 RPC 次数：每次 `create` 需要一次 `lookup`、一次 `create`、一次 `release` 共三次 RPC，RTT 约 700 μ s，加上 FUSE 调度开销后接近该数值。横向对比这一线的 $N \in \{2, 8, 32\}$ 三档继续沿用 $r = 1$ ，此时单 `rank` 尚未达到这一上限，比较的是各被测系统在相同请求速率下的吞吐。DeltaOp 内部对比这一线希望观察到服务端的事务冲突阈值，需要把到达 MDS 的请求速率继续推高。如果只是继续扩大 N ，每台客户端仍被限制在 400 ops/s 上限之下，到达服务端的总速率增长有限。因此在 $N \in \{64, 96\}$ 两档下，本章另外采用 $r = 2$ 与 $r = 4$ 两组设置：前者让每台客户端同时发起两条 FUSE 路径，后者为四条，这样可以在客户端数量不变的情况下把 np 继续向上推进。 $r = 4$ 时每台 2 核客户端上实际运行 4 个 `mdtest rank`，理论上存在两倍的 CPU 超分配；但 `mdtest` 绝大多数时间阻塞在 RPC 返回上，CPU 并不是瓶颈，实测 $r = 4$ 下 `per-client` 吞吐约为 $r = 1$ 的 1.8–1.9 倍，说明客户端侧仍有余量。具体的 N 与 r 组合见表 5-3。

表 5-3 性能测试档位矩阵

用途	被测系统	N	r	np
横向对比	四套系统均参与	2, 8, 32	1	2, 8, 32
RucksFS 扩展性	Delta 与 NoDelta	64, 96	1	64, 96
DeltaOp 内部对比	Delta 与 NoDelta	64	2, 4	128, 256
		96	2, 4	192, 384
NFS 缓存配置观察	NFS (启用默认 AC 配置)	2, 8	1	2, 8

每 rank 固定创建 2000 个文件，用于使单次 create/stat/remove 阶段有足够的采样时间（最大档位 np = 384 时总文件数达 768 000）。mdtest 的核心参数是 -F -C -T -r -i 3：-F 仅测试文件操作，-C/-T/-r 分别对应创建、查询、删除三个阶段，-i 3 表示每个阶段独立重复三次，取 Mean 作为主指标、Std Dev 作为噪声判据。hard 与 easy 的区别由 -u 参数控制：默认所有 rank 在同一父目录下操作（hard），加上 -u 后各 rank 在独立子目录下操作（easy）。本章默认讨论的都是 hard 模式的数据，easy 模式数据用于对照。

每次 mdtest 调用前，服务端和所有客户端都会同步执行 `echo 3 > /proc/sys/vm/drop_caches`，以清除上一轮遗留的页缓存。

本章涉及的全部编排脚本、各被测系统的切换与初始化脚本、mdtest 参数模板，以及下文各节所引数据的原始 mdtest 与 pjdftest 输出，均按被测系统和并发档位归档于本文配套仓库的 `testing/bench-v3/` 目录（仓库地址：<https://github.com/csjgg/rucksfs>）。

5.3.4 测试范围的几点说明

在查看具体数据之前，有几点需要先予以说明。本章的测试主要关注元数据路径的性能，并非对完整文件系统能力的全面评估。下列几项条件事先明确，便于后面数据的解读。

mdtest 使用空文件负载。 启动参数为 -w 0，每个文件在 create 之后直接 close，不写入任何数据。这样做是为了将元数据路径与数据路径分离，只考察元数据的表现。代价是 DataServer 基本不参与测试——它仅在 create 时分配一个空的 DataLocation，在 unlink 时接到一次“可以回收”的通知。真实混合读写负载下 read/write 的表现并不在本章关注范围之内。

DataServer 目前是最小实现。 如第 4.2 节所述，RawDiskDataStore 以固定偏移公式直接映射 inode，没有块映射，也没有空间回收，delete_data 目前仍是空操作。因此 unlink 的吞吐中有一部分来自“当前未执行真正的数据回收”这一事实，存在一定虚高，具体到 §5.5 节讨论 remove 时会再展开。

未提供副本容错。 RucksFS 目前的部署是单 MetadataServer 加单默认 DataServer，没有副本，也没有 Raft。JuiceFS+TiKV 即使部署为单节点，TiKV 内

部仍然运行 Raft 和 Percolator 两阶段提交^[39]，这些开销原本是为多副本和跨节点事务准备的。因此本章看到的 RucksFS 对 JuiceFS+TiKV 的性能优势，一部分源于省去了这些提交层开销，不能据此推出 RucksFS 作为一个完整的分布式系统在各方面均已超过 JuiceFS+TiKV。

在明确上述前提之后，下文的数据主要用于回答两个问题：其一，RucksFS 在元数据路径上相对 NFS 与 JuiceFS+TiKV 的横向位置；其二，DeltaOp 这一具体机制在高并发下究竟发挥了多大作用。

5.4 POSIX 合规性评估

POSIX 合规性这一线的测试在分布式部署下进行——客户端和 MetadataServer 位于不同节点，请求需要经过 gRPC 传输。一个用例能够通过，前提不仅是 MetadataServer 内部的事务逻辑正确，还要求错误码在穿过 gRPC 和 FUSE 两层之后依然保持不变。

5.4.1 pjdfstest 简介

`pjdfstest`^[40] 是业界常用的 POSIX 文件系统合规性测试套件，覆盖 `chmod`、`chown`、`link`、`mkdir`、`mkfifo`、`mknod`、`open`、`rename`、`rmdir`、`symlink`、`truncate`、`unlink`、`ftruncate` 等 13 类系统调用。套件中每条 POSIX 规则都会按文件类型 (`regular`、`dir`、`symlink`、`fifo`、`socket`、`char`、`block`) 展开成多条用例，目的是系统地验证同一条规则在各种对象上的行为一致性。本文采用 `pjdfstest` 的 Rust 实现版本，总共 398 条用例。

这种展开方式对被测系统较为严苛：同一条 POSIX 规则在不同文件类型上共享同一代码分支，但只要其中任意一种文件类型不被支持，整组用例都会被记为“失败”——即便被测系统对该规则的核心语义本身是正确的。因此直接使用总通过率衡量语义完整度并不合适，需要先划定“本文实现覆盖范围内”的用例集，再讨论结果。

5.4.2 实现范围与理论排除项

在给出测试数据之前，先根据第 3 章的设计边界把 398 条用例分为“在实现范围内”和“在实现范围外”两类。

排除项 A: mknod 与 mkfifo 相关用例。这两个接口管理的是 Unix 域 socket、命名管道、字符设备节点和块设备节点。它们的运行语义与内核设备号、内核管道缓冲区紧密绑定，与本文“通过键值对持久化命名空间”的研究主题正交；加之本文聚焦的负载（容器镜像、训练数据、日志、构建产物）基本不会涉及这类对象，实现这两个接口对论证元数据机制本身帮助不大。因此本文明确选择不实现这两个接口，RucksFS 对二者统一返回 EOPNOTSUPP（POSIX 规定的“操作不支持”错误码，用户程序收到该错误后会明确得到“文件系统不支持该操作”的结果，而不是被伪装为成功）。凡是直接调用 mknod/mkfifo 的用例，以及需要先用它们准备测试对象的派生用例，都属于这一类。举一个具体例子：pjdfstest 中“当 rename 的目标路径前缀不是目录时应返回 ENOTDIR”这条规则会按 7 种文件类型各生成一条用例，其中后 4 种需要先用 mknod/mkfifo 准备 fifo、socket 或设备节点；用例还未进入 rename 本身的语义检查即因 EOPNOTSUPP 而中断——而 RucksFS 在常规文件、目录、符号链接上对同一条规则都能给出符合规则的响应。

排除项 B: DataServer 单文件大小上限。作为最小实现，当前 RawDiskDataStore 使用固定偏移公式映射 inode，单文件逻辑上限为 64 MiB。open::interact_2gb 这条用例直接位于该上限之上，属于 DataServer 的开发期参数，与元数据设计所要回答的问题无关。

排除项 C: 与元数据设计无关的跳过项。pjdfstest 框架本身会跳过 48 条用例：27 条来自 utimensat 系列（需要单独的 feature flag，当前未启用），21 条涉及 remount（FUSE 配置不允许运行时重新挂载）和跨文件系统操作（测试环境未配置第二个文件系统）。这些项与 RucksFS 的元数据设计无关。

扣除上述三类理论排除项之后，剩下的即为“本文实现范围内应当通过”的用例集。表 5-4 给出分类汇总。

5.4.3 实现范围内的测试结果

在上述划定的 182 条“实现范围内”用例上，RucksFS 全部通过，通过率 100%。换言之，只要一条 POSIX 规则不涉及本文明确不实现的接口，RucksFS 在分布式部署条件下都给出了符合语义的响应。这一结果是本章后续性能数据

表 5-4 pjdftest 用例按本文实现范围的划分

类别	用例数	说明
实现范围内	182	本文选择实现的接口，应当全部通过
排除项 A (mknod/mkfifo 相关)	167	直接或前置依赖这两个接口的用例
排除项 B (单文件大小上限)	1	open::interact_2gb
排除项 C (框架跳过)	48	utimensat、remount、跨文件系统
合计	398	

的语义前提：后续的吞吐数据都建立在“实现范围内语义全部正确”的基础之上。

为了将结论落到具体数据上，表 5-5 按 13 类系统调用给出完整分布。表中 rename、unlink、link 等类别看起来失败数较高，但失败全部集中于前文所述“需要 mknod/mkfifo 准备测试对象”的派生用例上——每条规则展开的 7 种文件类型中，后 4 种在前置阶段即告中断。若按派生用例实际走到的代码路径折算，这几类在“实现范围内”同样是全部通过的。

表 5-5 pjdftest 各操作类别的结果分布

操作	通过	失败	跳过	合计	通过率
ftruncate	6	0	0	6	100.0%
truncate	14	4	1	19	73.7%
open	18	7	2	27	66.7%
chown	16	8	2	26	61.5%
rmdir	14	8	1	23	60.9%
mkdir	12	8	1	21	57.1%
symlink	12	11	1	24	50.0%
chmod	16	16	1	33	48.5%
mkfifo	9	11	1	21	42.9%
mknod	15	23	0	38	39.5%
rename	23	28	9	60	38.3%
unlink	13	20	1	34	38.2%
link	14	24	3	41	34.1%

5.4.4 与第 3 章设计决策的对照

仅凭“实现范围内 182 条全部通过”这一汇总数据，尚不足以说明 RucksFS 确实将第 3 章的若干设计决策落到了实处。为避免“因为未实现所以不会失败”这样的误读，下面挑选几条较易被忽视的边界语义进行说明。这些用例都位于实现范围内、都在通过列表之中，且每一条都可以直接对应到第 3 章的某个具体设计决策。

- **unlink::open_file_not_freed** (对应: 删除语义与句柄生命周期, 第 3.4 节)。文件被 unlink 之后, 若仍有打开句柄, 应当能够继续读写, 物理回收需推迟到最后一个句柄关闭。这要求 MetadataServer 同时维护 nlink 和打开计数: 当 nlink 归零但计数仍大于零时, 对象进入 pending_deletes, 由 release 触发最终清理。若事务未将“删除目录项”和“维持 inode 可达性”分开处理, 该用例将不通过。
- **rename::to_multiply_linked::regular** (对应: rename 覆盖逻辑, 算法 3-2)。rename 的目标若是一个带有多条硬链接的文件, 覆盖动作只能将目标的 nlink 减一, 而不能直接删除目标 inode。这一点对应第 3 章 rename 流程中“按覆盖删除逻辑处理目标对象”一步。
- **12 条 *::enametoolong_component 用例** (对应: 错误码穿透)。文件名长度超过 NAME_MAX 时应返回 ENAMETOOLONG。该约束在 MetadataServer 所有接受 name 字段的 RPC 入口统一校验, 错误码需经 gRPC 和 FUSE 两层传递后保持不变。
- **open::open_trunc** (对应: VfsCore 协调路径, 第 4.1 节)。以 O_TRUNC 打开已有文件时, size 需在打开操作中同步清零, 同时刷新 mtime 和 ctime。FUSE 并不一定会再发一次独立的 setattr, 因此这些工作必须在 open 实现中一并完成。
- **rmdir::eexist_enotempty_non_empty_dir 系列** (对应: 事务内空目录检查, 第 3.5 节)。删除非空目录应返回 ENOTEMPTY。第 3 章已经说明, rmdir 将空目录检查也纳入事务中, 在一致性视图下执行前缀扫描, 避免检查结果在进入事务前失效。

这些用例覆盖的并非孤立的接口行为, 而是第 3 章的跨列族原子提交、句柄生命周期、统一加锁顺序以及客户端协调路径在实际运行时的综合表现。它们能够全部通过, 说明这套设计在分布式部署条件下达到了预期的语义效果。

5.4.5 小结

综上所述: 在明确排除 mknod/mkfifo 相关用例、DataServer 单文件大小用例以及 pjdfstest 框架自身的跳过项之后, 剩下 182 条“实现范围内”用例全部通

过，通过率 100%。排除项中的 167 条失败由 `pjdfstest` 按文件类型展开的机制所放大，并不构成对元数据正确性的实际反例；`mknod/mkfifo` 的不实现是面向典型分布式存储负载所做的显式取舍，在返回值上亦被标记为 `EOPNOTSUPP`，而非伪装为“通过”。结合前述几条边界用例的对照，可以认为 RucksFS 的元数据实现在本课题要求的范围内达到了预期的语义正确性目标。这一结果同时也是后续性能数据的前提——后面的吞吐数据都建立在“实现范围内语义正确”的基础之上。

5.5 分布式性能对比

本节将 RucksFS-Delta 与 NFS、JuiceFS+TiKV 放在一起压测，在相同硬件、相同 `mdtest` 共享父目录 (`hard`) 模式下运行 `create`、`stat`、`remove` 三类操作。主要关心两件事：一是 RucksFS 这种基于 RocksDB 的元数据方案在性能上能否达到业界常见 KV 文件系统的水平，因此需要与 JuiceFS+TiKV 进行对比；二是相对传统本地文件系统，KV 这条路线是否带来了改善，因此也与 NFS 对比。下一节 (§5.6) 再单独对照 Delta 与 NoDelta，考察 DeltaOp 机制本身的效果。

在 N 较大 ($N \geq 64$) 时，两套对照系统都无法继续给出稳定的数据。JuiceFS+TiKV 方面，单节点 TiKV 在大量客户端同时挂载 FUSE 时会出现连接超时，生产环境本身也建议三节点起步，这里未补测。NFS 方面， $N = 32$ `hard` 模式下 `create` 的吞吐为 1733 ops/s，虽然相比 $N = 2$ 的 1038 ops/s 仍有缓慢增长，但与同档 `easy` 模式的 6445 ops/s 相比差距很大。共享父目录下的写路径已经被 `i_rwsem` 限制住，这一形态在更大并发下不会改变。因此下文主要讨论 $N \in \{2, 8, 32\}$ 三档的数据，该范围内三套系统都能给出可以相互比较的结果。 $N = 64$ 和 $N = 96$ 的 RucksFS 数据留到下一节讨论 DeltaOp 时使用。

此外需要说明一次 NFS 缓存配置的对照观察。本章默认以 `noac` 挂载 NFS，目的在实验目的一节已经说明。作为对照，本章另外在 $N \in \{2, 8\}$ 档位尝试以默认 AC 配置运行 `mdtest`。 $N = 2$ 时三次 iteration 的 `hard` 模式 `create` 吞吐分别为 1742、45、1164 ops/s，相对 Mean 的标准差达 83%，其中第二次 iteration 几乎停滞。 $N = 8$ 时 `mdtest` 在第一次 iteration 中即因 `open64` 返回 `ENOENT` 触发 `MPI_ABORT` 而终止。打印的错误路径是 `test-dir.0-0/mdtest_tree.0/file.mdtest.3.0`,

该文件确实由对应 rank 自行创建，但打开操作所在的客户端本地 dentry 缓存未及时刷新，被认定为“文件不存在”。NFSv4.2 默认 AC 挂载下，目录属性与目录项缓存的生存期由 acdirmin/acdirmax 决定（30–60 秒），多客户端并发写入同一目录时，协议本身不提供足以支撑这种访问模式的一致性保证。这一现象是 NFS 共享父目录场景的一项固有限制，也是本章选择 noac 作为 NFS 对比配置的直接原因。

5.5.1 原始测量结果

表 5-6–5-8 分别给出 create、stat、remove 三类操作在三档横向对比并发下的吞吐量。每个数值为 mdtest 三次 iteration 所取 Mean， $N \in \{2, 8, 32\}$ 三档下全部单元格的 Std Dev 均在 Mean 的 5% 以内，测量稳定。

表 5-6 文件创建吞吐量（hard 模式，单位 ops/s）

N	RucksFS-Delta	NFS	JuiceFS+TiKV
2	542	1,038	370
8	2,126	1,752	1,401
32	8,035	1,733	4,710

表 5-7 文件查询吞吐量（hard 模式，单位 ops/s）

N	RucksFS-Delta	NFS	JuiceFS+TiKV
2	678	1,661	562
8	2,626	6,142	2,200
32	10,007	24,696	8,022

表 5-8 文件删除吞吐量（hard 模式，单位 ops/s）

N	RucksFS-Delta	NFS	JuiceFS+TiKV
2	555	1,053	308
8	2,170	3,131	1,157
32	8,245	5,682	3,728

下面分别讨论 create、stat、remove 三类操作的表现。

5.5.2 创建吞吐

三套系统的 create 吞吐量对比见图 5-1。 $N = 2$ 时，三套系统的 hard 模式吞吐从大到小依次是 NFS (1 038)、RucksFS-Delta (542)、JuiceFS+TiKV (370)； $N = 8$ 时顺序变为 RucksFS-Delta (2 126)、NFS (1 752)、JuiceFS+TiKV (1 401)； $N = 32$ 时顺序变为 RucksFS-Delta (8 035)、JuiceFS+TiKV (4 710)、NFS (1 733)。NFS 从 $N = 8$ 到 $N = 32$ 反而略有下降。三条曲线的相对位置随 N 发生改变，是本节需要解读的主要现象。

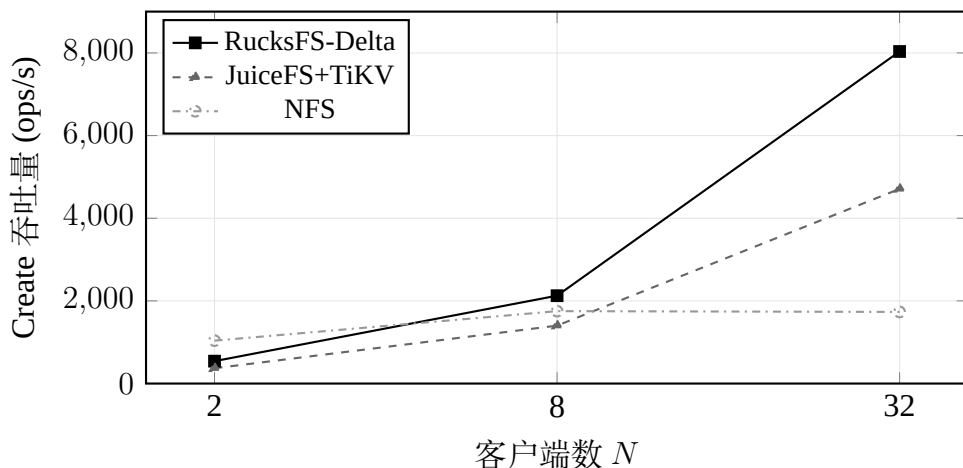


图 5-1 共享父目录场景下文件创建吞吐量随客户端数变化

与 JuiceFS+TiKV 的对比。RucksFS 在三档下对 JuiceFS+TiKV 的比值分别为 1.47、1.52、1.71，较为稳定。差距主要来自两方面。其一，RucksFS 采用单机直连 RocksDB，一次 create 在 MetadataServer 内部即一次 AtomicWriteBatch 提交，多类键共享一次 WAL 同步即可写入；而 TiKV 需要走完整的 Percolator 两阶段提交，再经过一遍 Raft 日志，即便单节点部署、没有跨节点复制，Raft 这一层也照样执行。这部分提交层开销正是 RucksFS 所省去的。其二，DeltaOp 机制将父目录属性更新从“读-改-写”改为对独立列族的追加写，使共享父目录下的 create 事务不再需要竞争父 inode 键的写锁， $N = 32$ hard 模式下 RucksFS 仍保持与 easy 模式接近的吞吐 (8 035 vs 8 028)。JuiceFS+TiKV 没有类似机制，共享父目录更新走 MVCC 的常规读写路径， $N = 32$ hard 模式相对 easy 模式为 4 710 vs 3 878，受并发影响在本档位不明显。

这一对比说明两件事。其一，RucksFS 基于 RocksDB 的元数据实现，在相

同硬件、相同客户端配置下 `create` 吞吐稳定高于业界常见的“通用分布式 KV + FUSE 文件系统”方案，整体上达到了生产可用的水平。其二，单机直连加针对共享热点目录的并发优化确实能够带来明显的性能提升，这正是本文的主要设计点。需要说明的是，此处 1.5–1.7 倍的差距是一个较为保守的数字：JuiceFS 对应的 TiKV 为单节点部署，已省去了多副本复制开销；若 TiKV 按生产环境推荐的三节点部署，这一比值还会更大。

与 NFS 的对比。 RucksFS 对 NFS 的比值从 $N = 2$ 的 0.52 上升到 $N = 8$ 的 1.21，再到 $N = 32$ 的 4.64。 $N = 2$ 时 NFS 领先，原因在于一次 NFSv4.2 `create` 在客户端只需一次 RPC 到服务端（COMPOUND 请求内可以把 LOOKUP 和 OPEN/CREATE 合并），而 RucksFS 在关闭 FUSE TTL 之后，每次 `create` 在客户端视角下会产生一次 `lookup`、一次 `create`、一次 `release` 共三次 RPC。这三次 RPC 是 VFS 语义所要求的三次事件，FUSE 框架不会合并。两侧单操作 RPC 次数之比为 1:3，与 $N = 2$ 时 1038:542 $\approx 1.9:1$ 的吞吐比大致吻合。当 N 增大时，NFS 服务端所有线程都要在父目录 inode 的 `i_rwsem` 写锁上排队，才能进入 `ext4` 进行目录项插入与 `journal` 提交： $N = 8$ 时 `hard/easy` 比为 0.48， $N = 32$ 时为 0.27，即共享父目录下的 NFS `create` 吞吐几乎不随 N 增长。RucksFS 的 `create` 吞吐在同一区间内接近线性扩展，两者差距从 0.52 倍翻转到 4.64 倍。

这一形态说明的事情是具体的：在共享父目录并发写入场景下，文件系统的主要瓶颈是父目录锁，而不是客户端与服务端之间的单次 RPC 开销。NFS 底层的 `ext4` 目录采用 `htree B+` 结构，写入必须串行；而 KV 方案可以把“目录项”和“父目录属性”拆成若干条独立键，用不同的事务路径处理，所以在高并发下还能继续扩展。这也是本文选择“以 KV 保存元数据”作为研究前提的直接原因。

5.5.3 查询吞吐

表 5-7 给出 `stat` 的吞吐数据。 $N \in \{2, 8, 32\}$ 三档下，三套系统的吞吐由高到低依次是 NFS（1661、6142、24696 ops/s）、RucksFS-Delta（678、2626、10007 ops/s）、JuiceFS+TiKV（562、2200、8022 ops/s）。 $N = 2$ 时 NFS 的三次 `iteration` 分别为 1660、2076、1247，标准差达到 274，波动较大，表中 1661 为三次的平均。

三套系统在 `stat` 上的顺序与 `create` 不同：`create` 下 RucksFS 居中 ($N = 2$) 或领先 ($N = 8, 32$)，而 `stat` 下 NFS 始终最高。这一差异来自两层原因：一是读取数据结构本身的效率，二是从客户端到服务端的路径开销。下面分别说明。

与 NFS 的差距：RucksFS 慢于 NFS 的事实。 三档下 NFS 的 `stat` 吞吐都稳定高于 RucksFS 约 2.3–2.5 倍。这一结果和 NFS 的 `create` 在 $N \geq 8$ 时反被 RucksFS 超过形成了对照——NFS 在读路径上有优势，写路径上则被共享目录锁限制住。本文诚实地承认这一事实：在关闭所有客户端缓存、每次 `stat` 都穿透到服务端的条件下，RucksFS 在读路径上确实慢于 NFS。

造成这一差距的原因有两层。第一层在路径开销。RucksFS 的一次 `stat` 要依次经过 FUSE 守护进程的系统调用返回、用户态 gRPC 客户端到服务端、MetadataServer 内部的 `getattr` 处理、RocksDB 点查，再返回折叠后的 inode 值；NFS v4.2 的 GETATTR 由内核直接处理，客户端走 kNFS 路径，服务端在内核上下文中读 ext4 目录 htree，没有用户态切换这一层开销。

第二层也是更本质的一层，在于存储结构本身。ext4 的目录 htree 是一棵 B+ 树的变体，点查从根节点沿哈希索引走到叶子节点，通常只需要 2–3 次页访问，整条查找路径上的指针解引用次数固定且很少。RocksDB 的 LSM-tree 则需要依次检查 MemTable、各层 SST 的布隆过滤器，只要某一层的布隆过滤器不确定，就要读一次对应层的 SST block。即便布隆过滤器命中率接近 100%、BlockCache 也命中，一次点查涉及的代码路径长度仍然高于 B+ 树直接走几层指针。这是 LSM 相对 B+ 树的固有代价：LSM 用更重的读路径换取更高的写吞吐。这一权衡在学术文献中已有清楚的讨论，本文在第 3 章选择 LSM 作为元数据引擎，正是因为本文所面向的工作负载中写操作（`create`、`mkdir`、`unlink` 等）出现的频次远高于需要通过点查完成的随机读，目录遍历也主要走前缀扫描而不是点查，LSM 的写吞吐收益大于其读路径上付出的代价。

因此 NFS 的 `stat` 吞吐高于 RucksFS 并不是实现不到位或配置不当，而是两种存储结构在读路径上各自擅长的体现。RucksFS 的 `stat` 吞吐可以做到随 N 近似线性增长（ $N = 2$ 到 $N = 32$ 约 15 倍，接近 16 倍的客户端数增长），这已经说明 InodeFoldedCache 加 BlockCache 两层缓存的配置在当前测试下起到了预期作用，使得 LSM 点查的额外开销没有进一步放大。

与 JuiceFS+TiKV 的差距。RucksFS 相对 JuiceFS+TiKV 略有领先 ($N = 32$ hard 模式下为 1.25 倍), 这一差距比 create 下的 1.71 倍有所缩小。原因在于读路径不涉及 Percolator 两阶段提交, TiKV 只需一次 BatchGet 加一次 read-index 即可返回。此时本文单机直连 RocksDB 的优势就只剩“本地调用对一次 gRPC”的差距。再加上本文在服务端显式维护了 InodeFoldedCache, 不必每次 getattr 都重新扫描 delta_entries, 因此仍保留了一定领先。

DeltaOp 对 stat 的代价。 DeltaOp 把父目录属性拆成“基值 + 若干条增量”, getattr 在读基值之后还要扫描 delta_entries 把增量叠加回去, 在路径长度上比直接读一条 inode 记录更长。这部分额外开销在 Delta 与 NoDelta 的对比中可以直接观察到: 下一节的数据显示, 两种构建在所有档位下的 stat 吞吐差距都落在标准差之内, 折叠代价在实测中观察不到。这一结果主要依赖 InodeFoldedCache 命中时直接返回已折叠结果, 以及 DeltaCompactionWorker 把增量条数控制在 32 条以内。

5.5.4 删除吞吐

remove 的数据见表 5-8。 $N \in \{2, 8, 32\}$ 三档下 RucksFS-Delta 分别为 555、2 170、8 245 ops/s; 对 JuiceFS+TiKV 的比值依次为 1.80、1.88、2.21, 整体与 create 相近且随 N 略有放大。对 NFS 的比值依次为 0.53、0.69、1.45, 和 create 一样呈现出“低并发 NFS 领先、高并发 RucksFS 反超”的形态——单次 RPC 次数与 ext4 目录锁同样是主导因素。

在解读上述数字前, 需要先说明一点: 当前 RucksFS 的 DataServer 仍是最小实现, delete_data 目前是空操作, 并不会进行真正的物理空间回收。相比之下, NFS 的 unlink 会触发 ext4 的 inode 位图翻转与 journal 提交, TiKV 的 unlink 需要写一条 tombstone 并最终参与后台 GC。mdtest 使用的又是空文件负载, 两侧本就没有实际需要回收, 因此这部分工作量差异在本文的测试中会被放大。所以 remove 的倍数中有一部分是“尚未进行数据回收”所带来的虚高, 并非全部源于设计上的收益。真实的数据回收补齐后, 这一倍数会相应回落。

另一方面, 元数据侧的 remove 路径在 RucksFS 中已完整实现。一次 unlink 在 MetadataServer 中同样走 AtomicWriteBatch 原子提交, 跨键更新按 inode 编

号排序加锁，与 `create` 对称；父目录属性的更新走 `DeltaOp` 追加，共享父目录下的并发 `unlink` 也不会因父 `inode` 键上互相抢锁。因此即便扣除前述虚高部分，元数据侧的 `remove` 路径仍能保持较高的并发度——下一节 `Delta` 与 `NoDelta` 对照中 `remove` 的分叉形态也可以印证这一点。

5.5.5 本节小结

本节的数据主要说明两件事。其一，本文基于 `RocksDB` 实现元数据，在相同硬件、相同客户端缓存配置下能够达到业界常见 `KV` 文件系统的水平——与 `JuiceFS+TiKV` 相比，`create/remove/stat` 三类操作在三档横向对比下均稳定领先，其中 `create` 1.5–1.7 倍、`remove` 1.8–2.2 倍、`stat` 约 1.2 倍；分项来看，主要差距来自单机直连 `RocksDB` 省去了 `Percolator` 与 `Raft` 两层提交开销，以及 `DeltaOp` 对共享父目录并发的优化。其二，相对传统本地文件系统，`KV` 这条路线在共享父目录高并发下更有扩展空间—— $N = 2$ 时 `NFS` 凭借单次 `RPC` 路径更短、`ext4` `B+` 树点查更快，仍略领先 `RucksFS`；到 $N = 32$ 时 `NFS` 被 `ext4` 的 `i_rwsem` 锁限制住，`create` 和 `remove` 均出现 `RucksFS` 反超——前者反超约 4.6 倍，后者约 1.5 倍。`stat` 方向则相反：`NFS` 始终领先 `RucksFS`，原因在于 `B+` 树点查与内核 `NFS` 路径在读取上更为直接。本文承认这一差距，不试图在读路径上反超。

横向对比只能覆盖 $N \leq 32$ 这一区间： $N \geq 64$ 时 `JuiceFS+TiKV` 单节点部署已接近连接规模的上限，`NFS` 在共享父目录 `hard` 模式下的 `create` 几乎不再随 N 增长。共享父目录高并发恰恰是 `DeltaOp` 机制要解决的场景。下一节通过 `Delta` 与 `NoDelta` 两种仅有一个编译开关差异的构建变体，在相同硬件、相同负载下进行对照，说明 `DeltaOp` 在这一区间发挥了多大作用。

5.6 DeltaOp 增量机制的内部对比

上一节主要讨论 $N \leq 32$ 的数据，原因是 $N \geq 64$ 时 `JuiceFS+TiKV` 和 `NFS` 均无法稳定运行。但高并发共享父目录恰好是当初设计 `DeltaOp` 机制所要针对的场景，因此本节专门考察 `DeltaOp` 在高并发下的实际作用。

做法是：保留主实现不动，仅通过一个 `cargo` 特性开关关闭 `DeltaOp`，得到一个名为 `NoDelta` 的构建。它与 `Delta` 的区别仅在父目录属性更新这一处——`NoDelta` 退回传统的“读父 `inode`、修改、写回”方式，其余代码完全一致。二者

在同一台服务器、同一批客户端、同一份负载下分别运行，直接比较。

本节需要先交代一个实际测量上的前提，这也是第 5.3.3 节所讨论的 rank 扩展机制在本节发挥作用的地方。关闭 FUSE 客户端属性缓存之后，单 rank 在 2 核客户端上的 create 吞吐上限约为 400 ops/s。 $N = 96$ 且每台客户端只启动 1 个 rank 时，到达服务端的请求速率约为 $96 \times 214 \approx 20\,500$ ops/s——本节第一组数据会看到，这一速率还没有触发 NoDelta 的事务冲突阈值。要让 DeltaOp 的作用显露出来，需要把服务端接收到的请求速率继续推高。为此本节使用 $N \in \{64, 96\}$ 与 $r \in \{1, 2, 4\}$ 的组合，共六个档位（np 分别为 64、96、128、192、256、384），沿并发度这一轴把整条曲线扫出来。

第 3 章第 3.4 节当时的设计思路是：将父目录属性更新从“读-改-写”改为向一个独立列族追加 DeltaOp，使热点父目录上的并发写入不再在同一条 inode 键上争抢写锁，从而避免高并发下的事务冲突。本节将数据按两条线组织：一条是沿 np 轴扫描，观察 Delta 与 NoDelta 的吞吐曲线是否在某个请求速率处分开；另一条是和 easy 模式做对照——easy 模式下每个 rank 在各自的子目录下操作，父目录键不再是写热点，若 Delta 相对 NoDelta 的差距确实来自父目录写冲突的消除，则 easy 模式下两者应当重合。两条线的数据分别见下。

5.6.1 完整的并发扫描

表 5-9 列出 Delta 与 NoDelta 在 hard 模式下沿 np 轴扫描的 create 吞吐，以及 easy 模式下的对照结果。

表 5-9 Delta 与 NoDelta 的 create 吞吐量扫描结果（单位 ops/s）

N	r	np	hard（共享父目录）			easy（独立父目录）		
			Delta	NoDelta	比值	Delta	NoDelta	比值
2	1	2	542	542	1.00	539	539	1.00
8	1	8	2,126	2,122	1.00	2,120	2,117	1.00
32	1	32	8,035	8,012	1.00	8,028	7,979	1.01
64	1	64	14,726	14,773	1.00	13,645	14,696	0.93
96	1	96	20,574	20,479	1.00	20,527	20,532	1.00
64	2	128	23,042	13,179	1.75	—	—	—
96	2	192	29,602	11,703	2.53	—	—	—
64	4	256	29,734	11,985	2.48	—	—	—
96	4	384	38,081	12,855	2.96	—	—	—

hard 模式下数据可以分两段观察。np ≤ 96 的五个档位下 Delta 与 NoDelta 的

比值均为 1.00，差距在标准差范围内；np 跨过 96 之后，从 np = 96 到 np = 128，NoDelta 的吞吐从 20 479 降至 13 179 ops/s（下降 36%），后续 192、256、384 三档都稳定在 11 700–12 900 区间；Delta 则从 20 574 依次上升到 29 602、29 734、38 081，np = 384 时仍保持扩展。两者的比值依次为 1.00、1.75、2.53、2.48、2.96。两段数据的形态和第 3 章的设计思路一致：低请求速率下没有父目录写冲突，是否启用 DeltaOp 结果相近；请求速率跨过某一阈值之后，NoDelta 的读-改-写路径在父 inode 键上出现写冲突，Delta 则不进入这一循环，两者差距被拉开。

easy 模式下 Delta 与 NoDelta 在所有档位的比值都在 1.00 附近， $N = 64$ easy 档 Delta 的一次 iteration 恰落在抖动较大的区间，是个别测量噪声，不影响主体判断。easy 模式让每个 rank 在各自的子目录下操作，父目录键不再是写热点——如果 Delta 相对 NoDelta 的优势来自其他原因（例如更好的内存布局或更高的 CPU 利用率等），那么 easy 模式下也应当观察到差距，而实际上没有。这组对照说明 hard 模式下的差距确实来自父目录写冲突，而非测试噪声或其他无关因素。

图 5-2 将 hard 模式的数据按 np 绘制为曲线，分叉形态更为直观。图中 $np \leq 96$ 的部分是 $r = 1$ 的扩展性曲线， $np > 96$ 的部分来自提高 r 值所带来的请求速率增量。

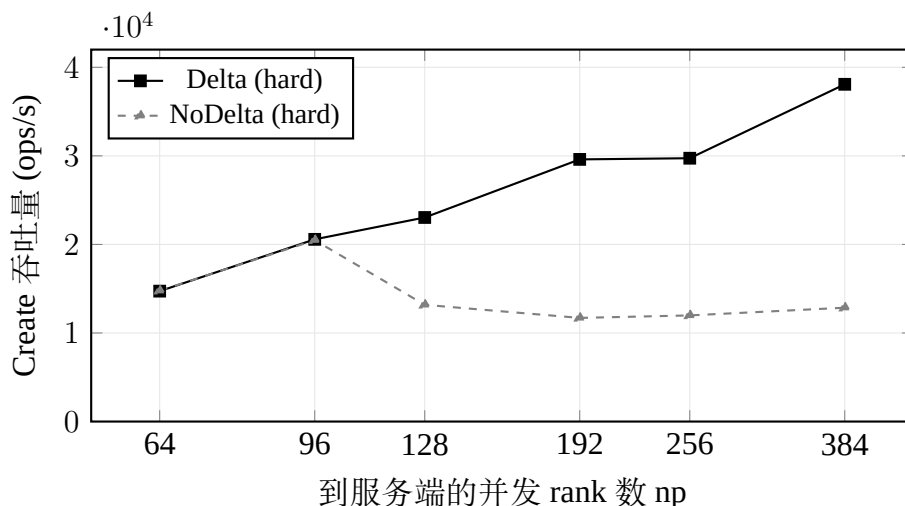


图 5-2 Delta 与 NoDelta 的 create 吞吐量对比（hard 模式，按 np 扫描）

5.6.2 NoDelta 为何在 np 跨过 96 后下降

NoDelta 的吞吐从 $np = 96$ 到 $np = 128$ 出现骤降、之后稳定在 12 000 ops/s 附近，这一现象的原因可以从 MetadataServer 端的事务执行过程中看出。

在 NoDelta 下，一次 create 事务内部需要完成的工作是：对父目录 inode 键调用一次 `get_for_update` 获取写锁，读取当前父 inode 的值，修改 `mtime/ctime`，再 `put` 回去，连同新建的目录项和 inode 一并提交。共享父目录下，所有客户端的 create 事务都在争抢同一条父 inode 键的写锁。如第 3 章所述，本文采用的是悲观并发控制：未取得锁的事务会等待，等待超时后回滚，由服务端再次重试。并发度上升后，这个“等待—回滚—重试”的循环会自我放大：被拒绝的事务需要先释放已经取得的其他锁、回滚已经写入的 batch，之后才能重新加入锁竞争；而此时队列中又堆积了更多重试请求。请求速率穿过某个阈值后，吞吐就会急剧下降，之后稳定在重试循环的稳态上限。

启用 DeltaOp 后，路径发生了明显变化。create 不再改动父 inode 的值，父目录的属性变化被编码为一条 DeltaOp，写入 `delta_entries` 列族。DeltaOp 的键中带有有一个单调递增的序列号 `seq`，每次写入的目标键互不相同，不会与其他事务在同一个键上争锁。父 inode 的值只在后台的 `DeltaCompactionWorker` 折叠增量时才被写一次，该动作不出现在测试的主要路径上，不影响 create 本身。锁竞争消除的同时，热点目录上的随机写也转为了顺序追加。

把上述原因对应到数据上： $np \leq 96$ 时锁等待还没有达到引发重试堆积的程度，NoDelta 与 Delta 吞吐基本一致； np 跨过 96 之后 NoDelta 吞吐降到 13 000 ops/s 左右，对应重试队列开始累积；再继续提高请求速率（192、256、384）时 NoDelta 仍稳定在这一区间，说明此时系统已进入重试循环的稳态——继续加压并不能让更多事务提交，反而让单次事务的实际耗时随重试次数而增加。Delta 不进入这一循环，因此吞吐能够一路扩展到 38 000 ops/s，并仍有上升趋势。

这一形态还解释了“客户端数 N ”与“DeltaOp 生效阈值”之间为什么看不到直接关系。对比 $N = 64, r = 2$ ($np = 128$) 与 $N = 96, r = 1$ ($np = 96$) 两组：前者客户端少，但 NoDelta 明显下降；后者客户端多，两者反而基本持平。真正决定事务冲突是否发生的，是服务端单位时间接收到的 create 请求数，不是发

起请求的客户端数量。本节沿 N 与 r 两个维度扫描的价值正在这里——把“到达服务端的请求速率”作为单独一个变量来控制，使得 DeltaOp 的作用可以直接对应到这一变量上。

5.6.3 对 remove 和 stat 的影响

DeltaOp 的设计目标虽然是父目录属性更新，但其影响并不仅限于 create。表 5-10 给出 remove 与 stat 沿 np 轴扫描下两种构建的对比。

表 5-10 DeltaOp 对 remove 与 stat 的影响 (hard 模式, 单位 ops/s)

N	r	np	remove			stat		
			Delta	NoDelta	比值	Delta	NoDelta	比值
64	1	64	14,239	15,208	0.94	18,413	18,505	1.00
96	1	96	21,673	21,311	1.02	25,743	25,869	0.99
64	2	128	24,996	13,507	1.85	29,624	29,823	0.99
96	2	192	31,836	11,374	2.80	37,718	35,820	1.05
64	4	256	27,847	12,521	2.22	39,719	39,551	1.00
96	4	384	32,882	12,500	2.63	51,613	49,746	1.04

remove 的形态与 create 相似。unlink 同样需要更新父目录的 mtime/ctime，走相同的事务流程。np ≤ 96 时 NoDelta 还未出现下降，跨过 96 之后立刻分开，稳态上限同样在 12 000 ops/s 左右。np = 192 时 remove 比值达到 2.80，稍高于同档 create 的 2.53。原因在于 unlink 事务在 NoDelta 下除了持父 inode 写锁，还需要对目标 inode 调用 get_for_update 以更新 nlink，持锁时间略长，重试堆积也相应更严重。

stat 在两种构建下吞吐一致。所有档位下 Delta 与 NoDelta 的 stat 吞吐比值都在 0.99–1.05 之间，差距小于标准差，两种构建表现相同。这一结果需要说明的是 DeltaOp 在读路径上的代价问题：启用 DeltaOp 之后，一次 getattr 在逻辑上除了读基值，还要扫描父目录 inode 对应的 delta_entries 并把尚未折叠的增量叠加回去，理论上会比直接读一个 inode 值慢。但数据显示这一代价在实测中观察不到。原因有两点：一是 InodeFoldedCache 把已经折叠好的

结果缓存下来，命中时一次内存查表即可返回，不触及 `delta_entries`；二是 `DeltaCompactionWorker` 把单个 `inode` 的增量条数控制在 32 以内，即便缓存未命中，叠加所需的扫描量也很小。也就是说，`DeltaOp` 给写路径带来的收益并没有在读路径上付出可以观察到的代价，这与第 3 章的设计目标一致。

5.6.4 回到第 3 章的设计

将上述几项结果与第 3 章关于 `DeltaOp` 的几条说法对照，整体上是一致的：

- **“`DeltaOp` 的收益在低并发下并不明显”**（第 3.4 节）。 $np \leq 96$ 时 `Delta` 与 `NoDelta` 在 `create`、`remove`、`stat` 三项上的比值均在 0.99–1.02 之间，基本相同。`DeltaOp` 针对的是热点父目录上的写争抢，没有形成争抢时，走读–改–写还是走增量追加结果相近。
- **“`DeltaOp` 在高并发下消除父目录事务冲突”**（第 3.4 节）。 np 从 96 跨到 128 时 `NoDelta` 出现明显的吞吐下降，之后在 12 000 ops/s 附近形成稳态；`Delta` 则继续扩展至 38 000 ops/s。两条曲线的分叉点正好对应 PCC 锁等待开始累积的位置。
- **“`DeltaOp` 的优势只在共享父目录场景下出现”**。`easy` 模式下两者吞吐基本一致，仅 `hard` 模式下存在差距。这也说明 `DeltaOp` 的收益并非来自其他无关因素。
- **`DeltaOp` 的折叠代价在读路径上观察不到**。`stat` 在两种构建下吞吐几乎相同，`DeltaOp` 没有让读路径变慢。这一点依赖 `InodeFoldedCache` 与 32 条增量上限两项配套设计，二者共同使得折叠动作的实际成本在当前测量下无法体现出来。

上述四点合起来可以说明，第 3 章所述的 `DeltaOp` 机制在分布式部署下确实发挥了作用：差异出现的方向与幅度、与请求速率的关系，以及 `easy/hard` 两种场景的对比，都与当初的设计吻合。

5.7 本章小结

本章的实验沿两条线展开。

第一条线是 POSIX 合规性。将 `pjdfstest` 的 398 条用例按本文实现范围分解：扣除 `mknod/mkfifo` 相关用例（167 条）、`DataServer` 单文件大小用例（1 条）和

pjdfstest 框架自身跳过的用例 (48 条) 之后, 剩下 182 条“实现范围内”用例全部通过, 通过率 100%。“打开后 unlink”、硬链接覆盖、长名称错误码、O_TRUNC 时间戳、事务内空目录检查等边界用例同样位于通过列表中, 正好对应第 3 章的跨列族原子提交、句柄生命周期维护、统一加锁顺序以及客户端协调路径。mknod/mkfifo 的不实现是面向典型分布式存储负载所作的显式取舍, 接口统一返回 EOPNOTSUPP, 不以错误来遮盖真实问题。

第二条线是分布式性能。横向对比主体数据来自 $N \in \{2, 8, 32\}$ 三档下对 NFS 与 JuiceFS+TiKV 的比较。与 JuiceFS+TiKV 相比, RucksFS 在 create/remove/stat 三类操作上均稳定领先——create 1.5–1.7 倍、remove 1.8–2.2 倍、stat 约 1.2 倍; 主要差距来自单机直连 RocksDB 省去了 Percolator 与 Raft 两层提交开销, 以及 DeltaOp 对共享父目录并发的优化。与 NFS 相比, $N = 2$ 时 NFS 凭借单次 RPC 路径更短与 ext4 B+ 树点查更快, create/remove/stat 三项均略领先 RucksFS; 随着 N 增大, i_rwsem 开始限制共享父目录下的写扩展, $N = 32$ 时 create 反超约 4.6 倍、remove 反超约 1.5 倍; stat 方向 NFS 始终领先, 本文承认这一存储结构上的差距, 不回避。横向对比只能覆盖 $N \leq 32$: $N \geq 64$ 时 NFS 与 JuiceFS+TiKV 各自面临不同层面的限制 (ext4 目录锁、单节点 TiKV 连接规模), 继续比较意义有限。

高并发区间的结果由 RucksFS 自身的 Delta 与 NoDelta 对比给出。两种构建仅差一个编译开关, 其余完全一致, 因此二者差距只能来自父目录更新路径的不同。为了覆盖事务冲突阈值前后的形态, 本章沿 np 轴扫描了六档: np ≤ 96 时两者持平, 都接近线性扩展; np 跨过 96 之后 NoDelta 进入“等待—回滚—重试”循环, 稳态吞吐停在 12 000 ops/s 附近; Delta 继续扩展到 38 000 ops/s 并仍有上升趋势。create/remove 方向上两者比值依次为 1.00、1.75、2.53、2.48、2.96 与 0.94、1.02、1.85、2.80、2.22、2.63; stat 方向上两者在所有档位都基本持平 (0.99–1.05), 说明 DeltaOp 的折叠代价在读路径上观察不到, InodeFoldedCache 与 32 条增量上限这两项配套设计使得折叠的实际成本在当前测量下无法体现出来。easy 模式下两者完全一致。这组对比说明: DeltaOp 在共享父目录高请求速率场景下确实消除了父 inode 的写冲突, 这正是第 3 章当时要解决的问题。

6 总结与展望

6.1 工作总结

本文围绕“基于 KV 存储管理文件系统元数据”这一具体问题，设计并实现了 RucksFS——一个以 RocksDB 为元数据引擎、通过 FUSE 对外提供 POSIX 接口的分布式文件系统原型，并完成了较为完整的实验评估。具体完成的工作如下：

- (1) 实现了基于 RocksDB 多列族的 FUSE 文件系统 RucksFS，支持 create、mkdir、unlink、rmdir、rename、readdir 等常用 POSIX 操作。元数据按访问模式划分到 inodes、dir_entries、delta_entries、system 四个列族；目录项键采用“父 inode + 子文件名”的大端序拼接编码，使文件操作基本对应于对目录树上一条边的增删。
- (2) 引入了增量更新与懒折叠机制 (DeltaOp)。父目录属性的修改不再就地重写基值，而是追加一条 DeltaOp，读路径在读取时再将基值和增量合成；后台线程在增量条数超过 32 时执行折叠。该机制借鉴了 Mantle 的 Delta Record 思路，但实现上更为轻量，整个流程都在单个 RocksDB 实例内完成。
- (3) 基于 RocksDB 悲观事务 (PCC) 处理并发。所有跨键的目录操作通过事务完成加锁与冲突检测；跨 inode 操作按编号排序加锁以避免死锁；临时冲突时服务端自动重试最多 3 次，失败后才向上层返回 EAGAIN。
- (4) 客户端基于 fuse3 + tokio 异步实现，挂载时启用 default_permissions 与 AllowOther，常规权限检查交由内核 VFS 完成。
- (5) 在 1 台 64 核服务器加最多 96 台 2 核客户端的腾讯云集群上完成了系统评估。POSIX 合规性方面，将 pjdftest 398 个用例按实现范围分解后，本文明确不实现的 mknod/mkfifo 相关用例 (167 条)、DataServer 单文件大小用例 (1 条) 和框架自身跳过项 (48 条) 合计 216 条；剩下 182 条“实现范围内”用例在分布式部署下全部通过，通过率 100%。性能方面分两段： $N \in \{2, 8, 32\}$ 三档下进行三方对比，RucksFS 相对 JuiceFS+TiKV 在 create/remove/stat 三项上分别领先约 1.5–1.7 倍、1.8–2.2 倍与 1.2 倍；相对 NFS 在 $N = 2$ 处因关

闭客户端缓存、单次 create 多走两次 RPC 而略低于 NFS，但到 $N = 32$ 时 ext4 的 `i_rwsem` 锁开始主导，RucksFS 在 create 上反超 NFS 约 4.6 倍；stat 方向 NFS 始终领先 RucksFS，原因是 ext4 B+ 树点查与内核 NFS 路径在读取上更为直接。 $N \geq 64$ 时 NFS 和 JuiceFS+TiKV 均无法给出稳定的数据，此时转用 Delta 与 NoDelta 两个构建变体互相对照，沿 $np = N \times r$ 两个维度扫描——np 跨过 96 后 NoDelta 进入事务冲突循环、稳态吞吐停在 12 000 ops/s 附近，Delta 则继续扩展至 38 000 ops/s，np = 384 时 create 比值 2.96、remove 比值 2.63；easy 模式下两者基本一致。

6.2 不足与展望

前述数据说明 RucksFS 在元数据关键路径上已经达到了当初设定的目标，但作为毕业设计阶段的原型，系统仍有若干明显不足。下列几项按优先级由高到低列出。

- (1) **gRPC 网络往返开销**。这是目前最直接的性能短板。分布式模式下，一次 create 需要多次跨节点 RPC，网络延迟占了相当大的比重。后续可以通过合并相邻 RPC 调用、引入客户端元数据缓存等方式加以缓解。
- (2) **DataServer 仍是最小实现**。当前 RawDiskDataStore 使用固定偏移公式直接映射 inode，`delete_data` 是空操作，不做空间回收，单文件存在 128 MiB 的逻辑上限。实验中使用 `mdtest` 的空文件负载可以跑通，但要作为真实可用的文件系统，还需要补上块分配、空间回收以及对大文件的支持。
- (3) **多节点扩展尚未实现**。当前实现虽然已经将 Client、MetadataServer、DataServer 分开部署，但主实现仍是单 MetadataServer 加单默认 DataServer，多节点元数据分片、数据副本以及故障切换均未实现。对应的架构预留（例如 `DataLocation` 中的路由字段）已经具备，但仍需后续完善。
- (4) **元数据预取与智能调度**。目前对所有元数据请求一视同仁，未区分冷热路径。实际负载中“`readdir` 完后立即批量 `stat`”较为常见，这类序列其实可以提前预取。后续可以探索利用 eBPF 在内核中采集访问模式，并据此实施预取。该方向的工程复杂度较高，建议排在前两项之后。

致 谢

落笔至此，四年的本科时光已近尾声。从最初面对一片空白的 `main.rs` 不知从何下手，到最终在真实分布式集群上把数字一行行跑出来，回望这段路，要感谢的人很多。

最先想到的自然是我的指导老师。初次与老师讨论课题时，我对 KV 存储与文件系统元数据之间的关系只有最朦胧的想法；是老师一次次把我从过于宏大的叙事拉回到具体的问题上——究竟要写哪条键？究竟该在事务里解决什么？究竟是 `DeltaOp` 起了作用，还是缓存起了作用？论文中的每一处改动、每一张表格、每一次结论的审慎措辞，背后都有老师的耐心批注与反复推敲。陆游有句诗，写系统工作再贴切不过——“纸上得来终觉浅，绝知此事要躬行”。本文若说有一点真切的体会，便是这一句。

也要感谢实验室的几位学长学姐。在我初次把代码跑出 `core dump` 时、在 `RocksDB` 事务冲突反复出现时、在远程测试脚本因为 `SSH` 配置问题一次又一次超时，他们都曾放下自己手头的工作来帮我看日志、讲原理、甚至直接替我改配置。许多细节在论文里已经化作了简短的一句说明，但这些时刻是真实存在过的，我也会记住。和他们的每一次讨论都让我意识到，科研不是一个人的事，一个好问题背后总是站着许多愿意一起思考的人。

感谢学校和学院四年来的培养。图书馆里安静的桌椅、实验课上第一次自己编译出的内核、深夜空无一人的机房、以及同学之间那些关于“这玩意到底能不能跑起来”的反复争论，共同构成了我工程素养的底色。大学并不只是知识的集合，更是一段把自己从“会做题的学生”慢慢变成“能解决问题的工程师”的过程。这一过程不会在毕业典礼那天戛然而止，但大学给了它最初的方向。

最后要深深感谢我的父母。写论文这段日子里，常常是通宵调代码到凌晨，白天又需要补睡再起来继续实验。父母从未催促过我什么，只是在每次视频时默默问一句“最近累不累”，在我难得回家的几天里准备好我爱吃的菜。他们或许并不完全理解 `inode`、`LSM-tree`、`PCC` 事务这些我口中的名词，但他们理解他们的孩子正在做一件自己认真对待的事情，这就足够了。他们的支持是我四年求学路上最稳的底，也是我接下来走入社会的底气。

走到今天，仍有许多未知在前方。但我也相信，只要还愿意在每一个具体的问题前坐下来、把一行字敲下去、把一次测试跑通，路就会一点一点延伸出来。谨以这篇论文，致我所遇见的每一位师长、同伴与家人。

参考文献

- [1] LI J, CAO B, JIAN J, et al. Mantle: Efficient Hierarchical Metadata Management for Cloud Object Storage Services[C]// Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP '25). [S.l.]: ACM, 2025:928–943. doi: 10.1145/3731569.3764824.
- [2] REN K, GIBSON G. TABLEFS: Enhancing Metadata Efficiency in the Local File System[C/OL]// 2013 USENIX Annual Technical Conference (USENIX ATC 13). San Jose, CA: USENIX Association, 2013: 145–156. <https://www.usenix.org/conference/atc13/technical-sessions/presentation/ren>.
- [3] O'NEIL P, CHENG E, GAWLICK D, et al. The Log-Structured Merge-Tree (LSM-Tree)[J]. Acta Informatica, 1996, 33(4): 351–385. doi: 10.1007/s002360050048.
- [4] DEAN J, GHEMAWAT S. LevelDB: A Fast Key-Value Storage Library. <https://github.com/google/leveldb>. Accessed: 2026. 2011.
- [5] DONG S, KRYCZKA A, JIN Y, et al. Evolution of Development Priorities in Key-Value Stores Serving Large-Scale Applications: The RocksDB Experience[C/OL]// 19th USENIX Conference on File and Storage Technologies (FAST 21). [S.l.]: USENIX Association, 2021: 33–49. <https://www.usenix.org/conference/fast21/presentation/dong>.
- [6] Apache Software Foundation. Apache Ozone: A Scalable, Redundant, Distributed Object Store for Hadoop. <https://ozone.apache.org>. Accessed: 2026. 2021.
- [7] PAN S, STAVRINOS T, ZHANG Y, et al. Facebook's Tectonic Filesystem: Efficiency from Exascale[C/OL]// 19th USENIX Conference on File and Storage Technologies (FAST 21). [S.l.]: USENIX Association, 2021: 1–16. <https://www.usenix.org/conference/fast21/presentation/pan>.

- [8] IEEE, The Open Group. IEEE Std 1003.1-2017 — POSIX.1-2017: Standard for Information Technology — Portable Operating System Interface (POSIX)[R/OL]. Standard. IEEE, 2017. <https://pubs.opengroup.org/onlinepubs/9699919799/>.
- [9] Juicedata, Inc. JuiceFS: A POSIX-Compatible Shared Filesystem for Cloud. <https://juicefs.com>. Open-source, available at <https://github.com/juicedata/juicefs>. 2021.
- [10] LIU H, WANG Q, YANG Y, et al. CFS: A Distributed File System for Large Scale Container Platforms[C]// Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19). [S.l.]: ACM, 2019. doi: 10.1145/3299869.3314046.
- [11] KLABNIK S, NICHOLS C. The Rust Programming Language. <https://doc.rust-lang.org/book/>. Also published by No Starch Press, ISBN 978-1-7185-0310-6. 2023.
- [12] REN K, ZHENG Q, PATIL S, et al. IndexFS: Scaling File System Metadata Performance with Stateless Caching and Bulk Insertion[C]// Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14). Best Paper Award. [S.l.]: IEEE Press, 2014: 237–248. doi: 10.1109/SC.2014.25.
- [13] LI S, LU Y, SHU J, et al. LocoFS: A Loosely-Coupled Metadata Service for Distributed File Systems[C]// Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17). [S.l.]: ACM, 2017: 1–12. doi: 10.1145/3126908.3126928.
- [14] XU Q, ARUMUGAM R V, YONG K L, et al. Efficient and Scalable Metadata Management in EB-Scale File Systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2014, 25(11): 2840–2850. doi: 10.1109/TPDS.2013.293.
- [15] GHEMAWAT S, GOBIOFF H, LEUNG S.-T. The Google File System[C]// Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03). [S.l.]: ACM, 2003: 29–43. doi: 10.1145/945445.945450.

- [16] SHVACHKO K, KUANG H, RADIA S, et al. The Hadoop Distributed File System[C]// 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST). [S.l.]: IEEE, 2010: 1–10. doi: 10.1109/MSST.2010.5496972.
- [17] WEIL S A, BRANDT S A, MILLER E L, et al. Ceph: A Scalable, High-Performance Distributed File System[C/OL]// Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06). [S.l.]: USENIX Association, 2006: 307–320. <https://www.usenix.org/conference/osdi-06/ceph-scalable-high-performance-distributed-file-system>.
- [18] DECANDIA G, HASTORUN D, JAMPANI M, et al. Dynamo: Amazon's Highly Available Key-Value Store[C]// Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07). [S.l.]: ACM, 2007: 205–220. doi: 10.1145/1294261.1294281.
- [19] GUO J, GUO L, SHI T, et al. SingularFS: A Billion-Scale Distributed File System Using a Single Metadata Server[C]// Proceedings of the 2023 USENIX Annual Technical Conference (ATC '23). [S.l.]: USENIX Association, 2023.
- [20] WANG Y, WU Y, LI C, et al. CFS: Scaling Metadata Service for Distributed File System via Pruned Scope of Critical Sections[C]// Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23). [S.l.]: ACM, 2023: 331–346. doi: 10.1145/3552326.3587443.
- [21] YANG Y, CAO Q, YANG L, et al. Batch-File Operations to Optimize Massive Files Accessing: Analysis, Design, and Application[J]. ACM Transactions on Storage, 2020, 16(3): 1–30. doi: 10.1145/3404119.
- [22] CAO X, PATEL S, LIM S.-Y, et al. FetchBPF: Customizable Prefetching Policies in Linux with eBPF[C]// Proceedings of the 2024 USENIX Annual Technical Conference (ATC '24). [S.l.]: USENIX Association, 2024.

- [23] TABATABAI B, III J C S, SWIFT M M. FBMM: Making Memory Management Extensible With Filesystems[C]// Proceedings of the 2024 USENIX Annual Technical Conference (ATC '24). [S.l.]: USENIX Association, 2024: 901–917.
- [24] HEIDARI A, AHMADI A, ZHI Z, et al. MetaHive: A Cache-Optimized Metadata Management for Heterogeneous Key-Value Stores. 2024. arXiv: 2407.19090 [cs.DB]. <https://arxiv.org/abs/2407.19090>.
- [25] SHEN J, ZUO P, LUO X, et al. FUSEE: A Fully Memory-Disaggregated Key-Value Store[C/OL]// 21st USENIX Conference on File and Storage Technologies (FAST 23). Santa Clara, CA: USENIX Association, 2023: 81–98. <https://www.usenix.org/conference/fast23/presentation/shen>.
- [26] DONG C, WANG F, YANG Y, et al. Low-Latency and Scalable Full-path Indexing Metadata Service for Distributed File Systems[C]// Proceedings of the 41st IEEE International Conference on Computer Design (ICCD 2023). [S.l.]: IEEE, 2023. doi: 10.1109/ICCD58817.2023.00051.
- [27] XU J, DONG M, TIAN Q, et al. SwitchFS: Asynchronous Metadata Updates for Distributed Filesystems with In-Network Coordination[C]// Proceedings of the 21st European Conference on Computer Systems (EuroSys '26). [S.l.]: ACM, 2026. doi: 10.1145/3767295.3769349.
- [28] 龚拓宇. 基于 RDMA 的集群文件系统元数据高速存取技术[D]. 清华大学, 2020.
- [29] 范立衡, 任祖杰. 基于键值存储的元数据集群副本一致性研究[J]. 杭州电子科技大学学报, 2014, 34(2): 89–93.
- [30] 吴雨飞, 李诚, 李嘉豪, 等. DFS 元数据缓存一致性的轻量级维护机制[J]. 计算机系统应用, 2025, 34(10): 1–8.
- [31] SZEREDI M. FUSE: Filesystem in Userspace. <https://github.com/libfuse/libfuse>. Linux kernel module since version 2.6.14. 2005.
- [32] BERNER C. Fuser: Filesystem in Userspace (FUSE) for Rust. <https://github.com/cberner/fuser>. Rust crate for FUSE, version 0.15. 2023.

- [33] VANGOOR B K R, TARASOV V, ZADOK E. To FUSE or Not to FUSE: Performance of User-Space File Systems[C/OL]// 15th USENIX Conference on File and Storage Technologies (FAST 17). Santa Clara, CA: USENIX Association, 2017: 59–72. <https://www.usenix.org/conference/fast17/technical-sessions/presentation/vangoor>.
- [34] ROSENBLUM M, OUSTERHOUT J K. The Design and Implementation of a Log-Structured File System[C]// Proceedings of the 13th ACM Symposium on Operating Systems Principles (SOSP '91). Also published in ACM Transactions on Computer Systems, 10(1):26–52, 1992. [S.l.]: ACM, 1992: 1–15. doi: 10.1145/121132.121137.
- [35] CHANG F, DEAN J, GHEMAWAT S, et al. Bigtable: A Distributed Storage System for Structured Data[C/OL]// Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06). [S.l.]: USENIX Association, 2006: 205–218. <https://www.usenix.org/conference/osdi-06/bigtable-distributed-storage-system-structured-data>.
- [36] LU L, PILLAI T S, ARPACI-DUSSEAU A C, et al. WiscKey: Separating Keys from Values in SSD-Conscious Storage[C/OL]// 14th USENIX Conference on File and Storage Technologies (FAST 16). [S.l.]: USENIX Association, 2016: 133–148. <https://www.usenix.org/conference/fast16/technical-sessions/presentation/lu>.
- [37] Meta Platforms, Inc. RocksDB Wiki. <https://github.com/facebook/rocksdb/wiki>. Accessed: 2026. 2023.
- [38] IOR/mdtest Development Team. Mdtest: HPC Metadata Benchmark. Part of the IOR project, used by IO500 for storage system ranking. 2024. <https://github.com/hpc/ior>.
- [39] PENG D, DABEK F. Large-Scale Incremental Processing Using Distributed Transactions and Notifications[C/OL]// Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10). [S.l.]:

USENIX Association, 2010: 1–15. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Peng.pdf.

- [40] DAWIDEK P J. Pjdfstest: POSIX File System Test Suite. Comprehensive POSIX compliance test suite for file systems. 2013. <https://github.com/pjd/pjdfstest>.